City University of Hong Kong
Department of Computer Science

CS3342 Software Design
Group Project Report

Course Registration System
Group 30

Lee Kwan Ho, Phelim 59284834

Yip Tung Yin, Sunny 58527052

Chen Kin Hang, Ivan 5928555

Chan Chun Hin, Isaac 59285357

Ieong Chun Ho, Nick 59286956

Ching Chun Hei, Martin 5930102

Table of Contents

Table of Contents 2
Summary and Requirements 6
Overview 6
Stakeholders 6
Objectives 6
Scope 8
Constraints 8
Functional Requirements 10
Non-functional Requirements 12
Reliability and Error handling 12
Security 12
Project Plan 13
Software Development Methodology 13
Work Breakdown Structure 13
Project Schedule 14
Development Tools 14
Analysis and Design 15
Use Case Diagram 15
Use Case Specification 16
Login 16
View Master Class Schedule 17
Manage Plans 18
Create Plan 19
Modify Plan 20

Remove Plan 21
Add Section 22
Drop Section 23
Join Waitlist 24
Drop Waitlist 25
View Timetable 26
Export Timetable 27
View Student List 28
Export Student List 29
Manage Courses 30
Create Course 31
Modify Course 32
Remove Course 33
Manage Registration Period 34
Manage Users 35
Create User 36
Modify User 37
Sequence diagram 38
Login 38
View master class schedule 39
Manage Plan 40
Create Plan 41
Modify Plan 42
Remove Plan 43
Add Section 44
Drop section 45

Drop Waitlist 46
Check Timetable 47
Export Timetable 48
Check Student List 49
Export Student List 50
Manage Course 51
Create Course 52
Modify Course 53
Remove Course 54
Manage Registration Period 55
Manage User 56
Create User 57
Modify User 58
Remove User 59
Implementation 60
System Prototype 62
Command Line Interface showcase 62
Web Interface showcase 64
File Persistence 68
Class Diagram 69
Software Design Principle and Patterns 69
Dependency Injection and Testability 69
Strategy Pattern // Todo: update to reflect timetable code change 70
Observer Pattern 71
Adapter Pattern 72
Facade Pattern 72

Builder Pattern 73
Software Testing 74
RegistrationService.addSection(): 76
TimetableService.exportTimetable(): 76
RegistrationService.waitListSection(): 77
Weekly Activity and Contribution 79
Work Allocation 79
Weekly Time Sheet and Activity 79
GitHub Activity Evidence 87
Conclusion 89

Summary and Requirements

Overview

This project implements a course registration system designed to help university members manage course enrollment for the upcoming semester, so students can register or remove courses, students and instructors can view and export their timetables, and administrators can manage course offerings while preserving valid section capacity and registration rules.

Implemented with a command line interface and with support for plain text data persistence, the system serves as a demonstration good understanding and application of the modern software engineering process with agile development lifecycle, requirement analysis and modeling, and object-oriented design.

Stakeholders

Actors	Responsibilities
User	Access the system
Student	Manage their course registration, access timetable
Instructor	Access timetable and student lists
Admin	Manage course offerings, users, registration periods

Objectives

- Design and develop a course registration system that supports essential academic enrollment activities for university stakeholders.
- Improve registration planning and timetable visibility for students, enabling informed decision-making and reducing manual trial-and-error during course registration with responsive user interface feedback.
- Enforce academic rules and constraints consistently throughout the registration process.

- Design a maintainable, extensible, and testable system by applying object-oriented design principles and proper design patterns.
- Demonstrate a clear understanding and practical application of software engineering tools, techniques, and procedures.

Scope

The system is designed for managing course registration within a single academic term.

It allows

- User authentication via login.
- Students to browse course offering, add and drop course sections, manage their registration plans, and view and export their timetables.
- Instructors to view and export timetables, and lists of enrolled students of sections they are responsible for.
- Administrators to manage registration periods, users, and course details such as section capacity.

The current system design does **not** cover

- Procedures that would require special approvement from the university, such as study load adjustment requests.
- Handling any task that is not a part of course registration, such as grading, attendance tracking, or tuition payment.
- Integration with university SSO authentication.
- Real-time concurrent registration handling.

Constraints

The system must be implemented as a Java application. The system must provide a Command Line interface. The system must support data persistence implemented using simple file input/output such as text, CSV, or JSON files. The system must be implemented within 1 semester, by a team of 6 students.

Platform	Java
Interface	Command Line Interface
Data Persistence	Simple File I/O (txt/csv/json)
Time	1 semester

Team size	6 students

Functional Requirements

The system should allow users to log into the system as their role, given that an account exists in the system, and that they are not logged in already. The system should only allow users to perform action available to their role.

The system should allow students to view the master class schedule. The system should allow students to manage their registration plan by creating, modifying, or removing them. The system should allow students to add and drop sections. The system should allow students to Join and Drop Waitlist.

The system should allow students and instructors to view and export Student List.

The system should allow instructors to view and export their timetables.

The system should allow administrators to manage courses by creating, modifying, or removing them in the system. The system should only allow courses with no enrolled students to be removed. The system should allow administrators to manage users by creating or modifying them in the system. The system should allow administrators to manage the registration period of each cohort of students.

Actors	Use Case
User	Login
Student	View Master Class Schedule
Student	Manage Plans
Student	Create Plans
Student	Modify Plans
Student	Remove Plans
Student	Add Section
Student	Drop Section
Student	Join Waitlist
Student	Drop Waitlist

Student, Instructor	View Timetable
Student, Instructor	Export Timetable
Instructor	View Student List
Instructor	Export Student List
Admin	Manage Course
Admin	Create Course
Admin	Modify Course
Admin	Remove Course
Admin	Manage Registration Period
Admin	Manage User
Admin	Create User
Admin	Modify User

Non-functional Requirements

Reliability and Error handling

- The system should gracefully handle invalid inputs and provide meaningful error messages
- The system should not crash and should present a stable user interface after an error
- The system should maintain correct state and revert changes after invalid operations

Security

- The system should correctly enforce role-based access
- The system should not reveal internal states when authentication fails

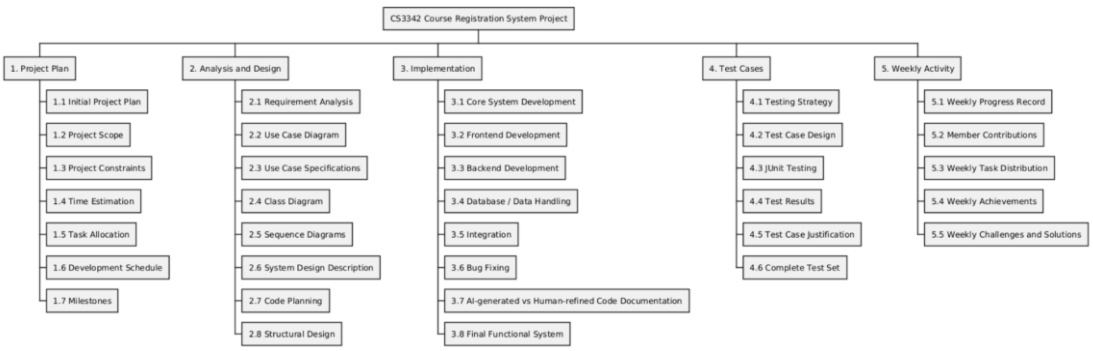
Project Plan

Software Development Methodology

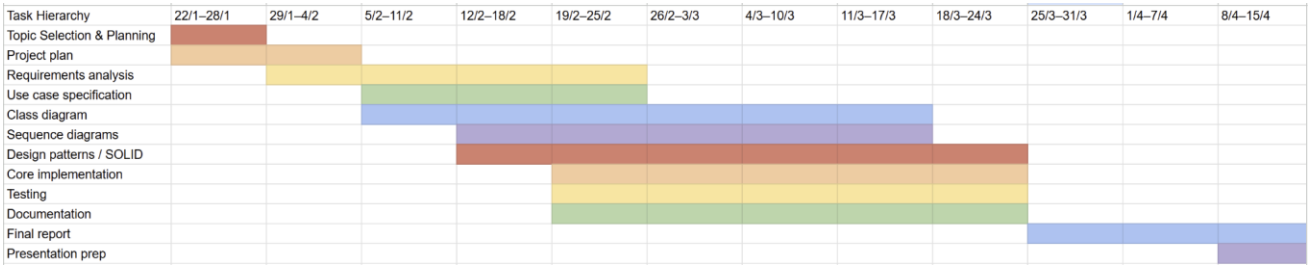
To develop the Course Registration System effectively, we plan to practice agile development practices by:

- Dividing the project into milestones and small sprints, each focusing on a few functional features.
- Using user stories to capture the needs of students, instructors, and administrators.
- Conducting continuous testing with JUnit and JaCoCo to ensure each iteration delivered working software.
- Practicing continuous integration (CI) using GitHub Actions for automated styling and testing.

Work Breakdown Structure



Project Schedule

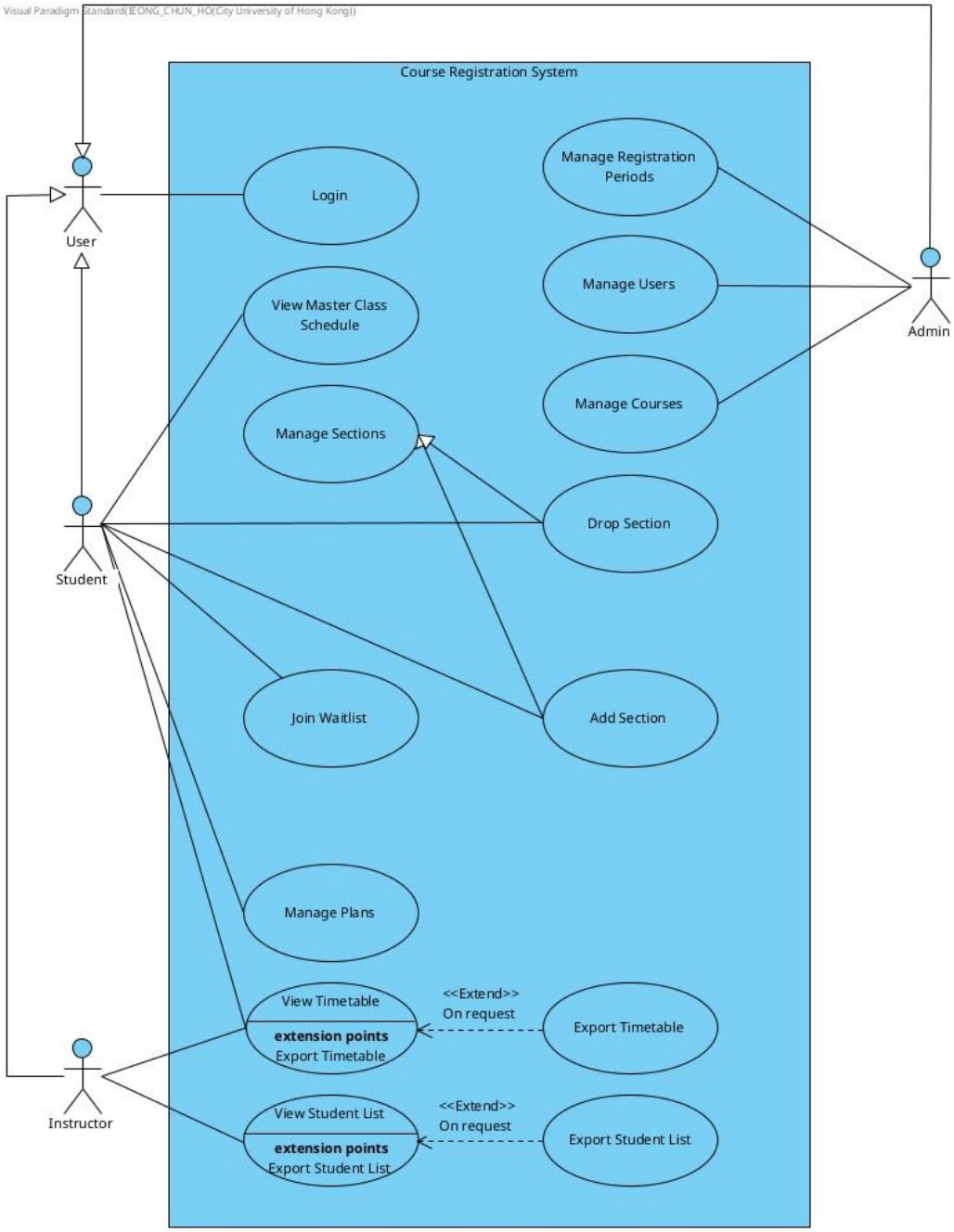


Development Tools

- Java 17
- Maven, Eclipse IDE v4.38.0, and Spring framework v4.0.2 for Java development
- Visual Paradigm v18.0 for UML modelling
- Git and GitHub for version control
- JUnit5 and Mockito v5.20.0 for unit testing
- JaCoCo v0.8.11 for test coverage
- GitHub Actions for continuous integration (CI)

Analysis and Design

Use Case Diagram



Use Case Specification

Login

Use Case Name:	Login	
Actor(s):	User	
Description:	This use case describes the process of a user logging in to the Course Registration System.	
Typical Course of Events:	**Actor Action** 1. User requests to login. 3. User submits credentials.	**System Response** 2. System prompts the user to enter credentials. 4. System validates the inputs. 5. System authenticates the user.
Alternate Courses of events:	3a. If the user requests to stop the use case, the use case ends. 4a. If the user leaves required fields empty, the system highlights the missing fields, presents an error message, and prompts the user to enter credentials. Rejoin Step 3. 5a. If authentication fails due to wrong credentials, the system presents an error message to notify the user and prompts the user to retry. Rejoin Step 3.	
Precondition:	The user is not logged in. The user has a registered account.	
Postcondition:	The user is logged in. The system has created a new session for the user.	

View Master Class Schedule

Use Case Name:	View Master Class Schedule	
Actor(s):	Student	
Description:	This use case describes the process of a student viewing the master class schedule	
Typical Course of Events:	**Actor Action** 1. Student request to view the master class schedule. 4. Student selects filters for courses.	**System Response** 2. System retrieves all courses. 3. System presents the courses. 5. System applies the filters and presents the updated list of matching courses.
Alternate Courses of events:	5a. If the student modifies the filters, the system applies the modified filters and presents the updated list of matching courses.	
Precondition:	The student is logged in.	
Postcondition:	The system has presented to the student courses matching their filters on the master class schedule as requested.	

Manage Plans

Use Case Name:	Manage Plans	
Actor(s):	Student	
Description:	This use case describes the process of a student creating, modifying, or removing a course registration plan.	
Typical Course of Events:	**Actor Action** 1. Student requests to manage their plans. 3. Student chooses to create, modify, or remove a plan.	**System Response** 2. System presents their existing plans. 4. System invokes the corresponding use case, Create Plan, Modify Plan, or Remove Plan, to perform the action.
Alternate Courses of events:	2a. If the student does not have a plan, the system invokes the use case Create Plan.	
Precondition:	The student is logged in. The registration period is not open to the student.	
Postcondition:	The system has created, modified, or removed the student's registration plans as requested.	

Create Plan

Use Case Name:	Create Plan	
Actor(s):	Student	
Description:	This use case describes the process of a student creating a course registration plan.	
Typical Course of Events:	**Actor Action** 1. Student requests to create a plan.	**System Response** 2. System creates a plan for the student. 3. System invokes the Modify Plan use case.
Alternate Courses of events:	N/A	
Precondition:	The student is logged in. The registration period is not open to the student.	
Postcondition:	The system has created a new course registration plan for the student as requested.	

Modify Plan

Use Case Name:	Modify Plan	
Actor(s):	Student	
Description:	This use case describes the process of a student modifying a course registration plan.	
Typical Course of Events:	**Actor Action** 1. Student requests to modify a plan. 3. Student modifies the plan by adding or dropping sections.	**System Response** 2. System presents the plan details. 4. System validates the request. 5. System submits the modified plan. 6. System presents the updated plan.
Alternate Courses of events:	4a. If a change is invalid, the system presents an error message to notify the student and prompts the student to perform alternative actions. Rejoin Step 3. 7. [Extension point: If the registration period is open to the student, the system invokes the Submit Plan use case]	
Precondition:	The student is logged in. The registration period is not open to the student.	
Postcondition:	The system has modified and saved the student's course registration plan as requested.	

Remove Plan

Use Case Name:	Remove Plan	
Actor(s):	Student	
Description:	This use case describes the process of a student removing a course registration plan.	
Typical Course of Events:	**Actor Action** 1. Student requests to remove a plan. 3. Student confirms the removal.	**System Response** 2. System prompts the student for confirmation. 4. System removes the plan.
Alternate Courses of events:	N/A	
Precondition:	The student is logged in. The registration period is not open to the student.	
Postcondition:	The system has removed the student's course registration plan as requested.	

Add Section

Use Case Name:	Add section	
Actor(s):	Student	
Description:	This use case describes the process of a student adding a section for enrollment.	
Typical Course of Events:	**Actor Action** 1. Student requests to add a section.	**System Response** 2. System validates the request. 3. System enrolls the student to the section and notifies the student.
Alternate Courses of events:	N/A	
Precondition:	The student is logged in. The registration period is open to the student. The section is not full. The student has fulfilled the course prerequisites of the section. The student has not taken the exclusive courses of the course of the section. The student is not enrolled to the section. The section does not have time conflict with the registered sections of the student.	
Postcondition:	The system has enrolled the student to the section as requested.	

Drop Section

Use Case Name:	Drop Section	
Actor(s):	Student	
Description:	This use case describes the process of a student dropping an enrolled section.	
Typical Course of Events:	**Actor Action** 1. Student requests to drop a section.	**System Response** 2. System validates the request. 3. System removes the student from the section.
Alternate Courses of events:	2a. If the section has linked sections that the student is enrolled to, the system prompts the student for confirmation to invoke the use case Drop Section for each of the linked sections and rejoin step 2. Otherwise, the use case ends.	
Precondition:	The student has logged in. The registration period is not open to the student. The student is enrolled to the section.	
Postcondition:	The system has removed the student from the section as requested.	

Join Waitlist

<<extends>> from add section

Use Case Name:	Join Waitlist	
Actor(s):	Student	
Description:	This use case describes the process of a student joining the waitlist for a full section.	
Typical Course of Events:	**Actor Action** 1. Student requests to join a waitlist.	**System Response** 2. System queues the student to the waitlist of the section.
Alternate Courses of events:	N/A	
Precondition:	The student is logged in. The registration period is open to the student. The section is full and the waitlist is not full. The student is not enrolled to the section.	
Postcondition:	The system has queued the student into the waitlist as requested.	

Drop Waitlist

Use Case Name:	Drop Waitlist	
Actor(s):	Student	
Description:	This use case describes the process of a student leaving the waitlist of a section.	
Typical Course of Events:	**Actor Action** 1. Student requests to leave a waitlist.	**System Response** 2. System removes the student from the waitlist of the section.
Alternate Courses of events:	N/A	
Precondition:	The student is logged in. The registration period is open to the student. The student is in the waitlist the section. The student is not enrolled to the section.	
Postcondition:	The system has removed the student from the waitlist as requested.	

View Timetable

Use Case Name:	View Timetable	
Actor(s):	Student, Instructor	
Description:	This use case describes the process of an instructor or a student Viewing the timetable.	
Typical Course of Events:	**Actor Action** 1. Actor requests to view the timetable.	**System Response** 2. System retrieves and presents the timetable.
Alternate Courses of events:	3. [Extension point: If the actor requests to export the timetable, invoke the use case Export Timetable].	
Precondition:	The actor is logged in.	
Postcondition:	The system presents the timetable to the actor as requested.	

Export Timetable

<<extends>> from View timetable

Use Case Name:	Export Timetable	
Actor(s):	Student, Instructor	
Description:	This use case describes the process of an instructor or a student exporting the timetable.	
Typical Course of Events:	**Actor Action** 1. Actor requests to export the timetable. 4. Actor downloads the file.	**System Response** 2. System writes the timetable to a file. 3. System presents the file to the actor for download.
Alternate Courses of events:	N/A	
Precondition:	The actor is logged in. The use case was invoked from the View Timetable use case.	
Postcondition:	The system exports the timetable for the actor to download as requested.	

View Student List

Use Case Name:	View student list	
Actor(s):	Instructor	
Description:	This use case describes the process of an instructor viewing the student list of their courses.	
Typical Course of Events:	**Actor Action** 1. Instructor requests to view the student list.	**System Response** 2. For each course the instructor teaches, system retrieves and presents a list of all students registered to the course.
Alternate Courses of events:	3. [Extension point: If the instructor requests to export the student list, invoke the use case Export Student List].	
Precondition:	The instructor is logged in.	
Postcondition:	The system presents the student lists to the instructor as requested.	

Export Student List

<<extends>> from view student list

Use Case Name:	Export student list	
Actor(s):	Instructor	
Description:	This use case describes the process of an instructor exporting the student list of their courses.	
Typical Course of Events:	**Actor Action** 1. Instructor requests to export student list. 4. Instructor downloads the file.	**System Response** 2. For each course the instructor teaches, the system writes a list of all students registered to the course to a file. 3. System presents the file to instructor for download.
Alternate Courses of events:	N/A	
Precondition:	The instructor is logged in. The use case was invoked from the View Student List use case.	
Postcondition:	The system exports the student list for the instructor to download as requested.	

Manage Courses

Use Case Name:	Manage Courses	
Actor(s):	Admin	
Description:	This use case describes the process of an admin managing courses by creating, modifying or removing them.	
Typical Course of Events:	**Actor Action** 1. Admin requests to manage courses. 3. Admin chooses to create, modify, or remove a course.	**System Response** 2. System presents the list of courses. 4. System invokes the corresponding use case, Create Course, Modify Course, or Remove Course, to perform the action.
Alternate Courses of events:	N/A	
Precondition:	The admin is logged in.	
Postcondition:	The system has created, modified, or removed the courses in the system as requested.	

Create Course

Use Case Name:	Create Course	
Actor(s):	Admin	
Description:	This use case describes the process of an admin creating a course.	
Typical Course of Events:	**Actor Action** 1. Admin requests to create a course.	**System Response** 2. System creates a course. 3. System invokes the Modify Course use case.
Alternate Courses of events:	N/A	
Precondition:	The admin is logged in.	
Postcondition:	The system has created a new course in the system as requested.	

Modify Course

Use Case Name:	Modify Course	
Actor(s):	Admin	
Description:	This use case describes the process of an admin modifying the details of a course.	
Typical Course of Events:	**Actor Action** 1. Admin requests to modify a course. 3. Admin modifies the details of the course. 6. Admin requests to save the course details.	**System Response** 2. System presents the course details. 4. System validates the changes. 5. System presents the updated course details. 7. System saves the modified course.
Alternate Courses of events:	4a. If a change is invalid, the system presents an error message to notify the admin and prompts the admin to perform alternative actions. Rejoin Step 3. a. During step 3 to 6, If the admin requests to stop modifying and has made changes to the course details, the system prompts the admin to save the changes. If the admin chooses to save, rejoin step 4. If the admin confirms to leave, the use case ends.	
Precondition:	The admin is logged in. No student is enrolled to the course.	
Postcondition:	The system has modified and saved the course details as requested.	

Remove Course

Use Case Name:	Remove Course	
Actor(s):	Admin	
Description:	This use case describes the process of an admin removing a course.	
Typical Course of Events:	**Actor Action** 1. Admin requests to remove a course. 3. Admin confirms the removal.	**System Response** 2. System prompts the admin for confirmation. 4. System removes the course.
Alternate Courses of events:	N/A	
Precondition:	The admin is logged in. No student is enrolled to the course.	
Postcondition:	The system has removed the course from the system as requested.	

Manage Registration Period

Use Case Name:	Manage registration Period	
Actor(s):	Admin	
Description:	This use case describes the process of an admin managing the registration periods of each cohort.	
Typical Course of Events:	**Actor Action** 1. Admin requests to manage registration periods. 3. Admin modifies the periods.	**System Response** 2. System presents a list of all cohorts and their registration periods. 4. For each cohort, the system validates the period. 5. For each cohort, the system updates the registration period. 6. System presents the updated list of periods.
Alternate Courses of events:	4a. If a period is invalid, the system presents an error message to notify the admin and prompts the admin to retry. Rejoin step 3.	
Precondition:	The admin is logged in.	
Postcondition:	The system has updated the list of periods set by the admin as requested.	

Manage Users

Use Case Name:	Manage Users	
Actor(s):	Admin	
Description:	This use case describes the process of an admin creating, modifying, or removing a user.	
Typical Course of Events:	**Actor Action** 1. Admin requests to manage the users. 3. Admin chooses to create, modify, or remove a user.	**System Response** 2. System presents the list of users in the system. 4. System invokes the corresponding use case, Create User, Modify User, to perform the action.
Alternate Courses of events:	N/A	
Precondition:	The admin is logged in.	
Postcondition:	The system has created, modified, or removed the user in the system as requested.	

Create User

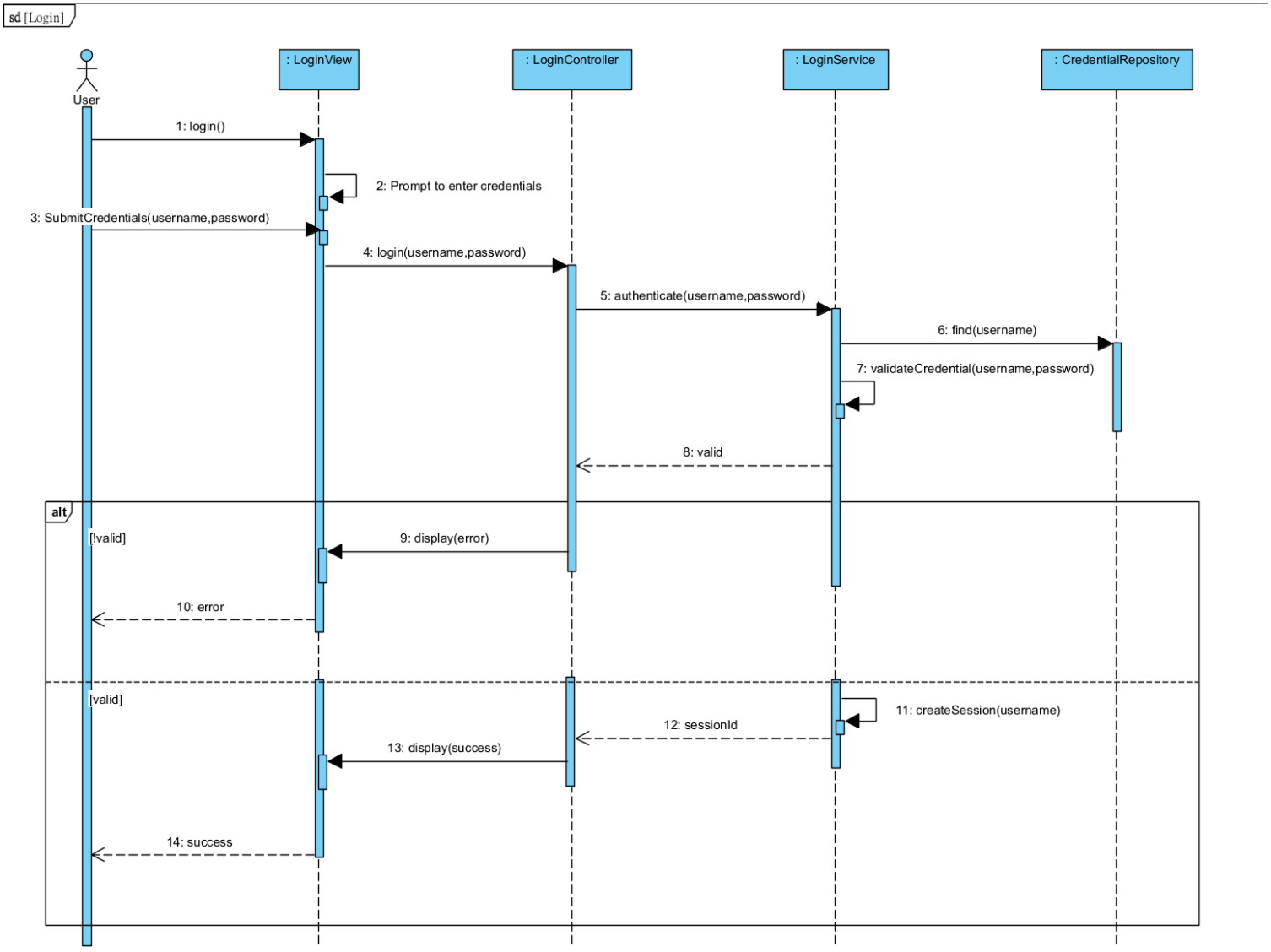
Use Case Name:	Create User	
Actor(s):	Admin	
Description:	This use case describes the process of an admin creating a user.	
Typical Course of Events:	**Actor Action** 1. Admin requests to create a user.	**System Response** 2. System creates a user. 3. System invokes the Modify User use case.
Alternate Courses of events:	N/A	
Precondition:	The admin is logged in.	
Postcondition:	The system has created a new user in the system as requested.	

Modify User

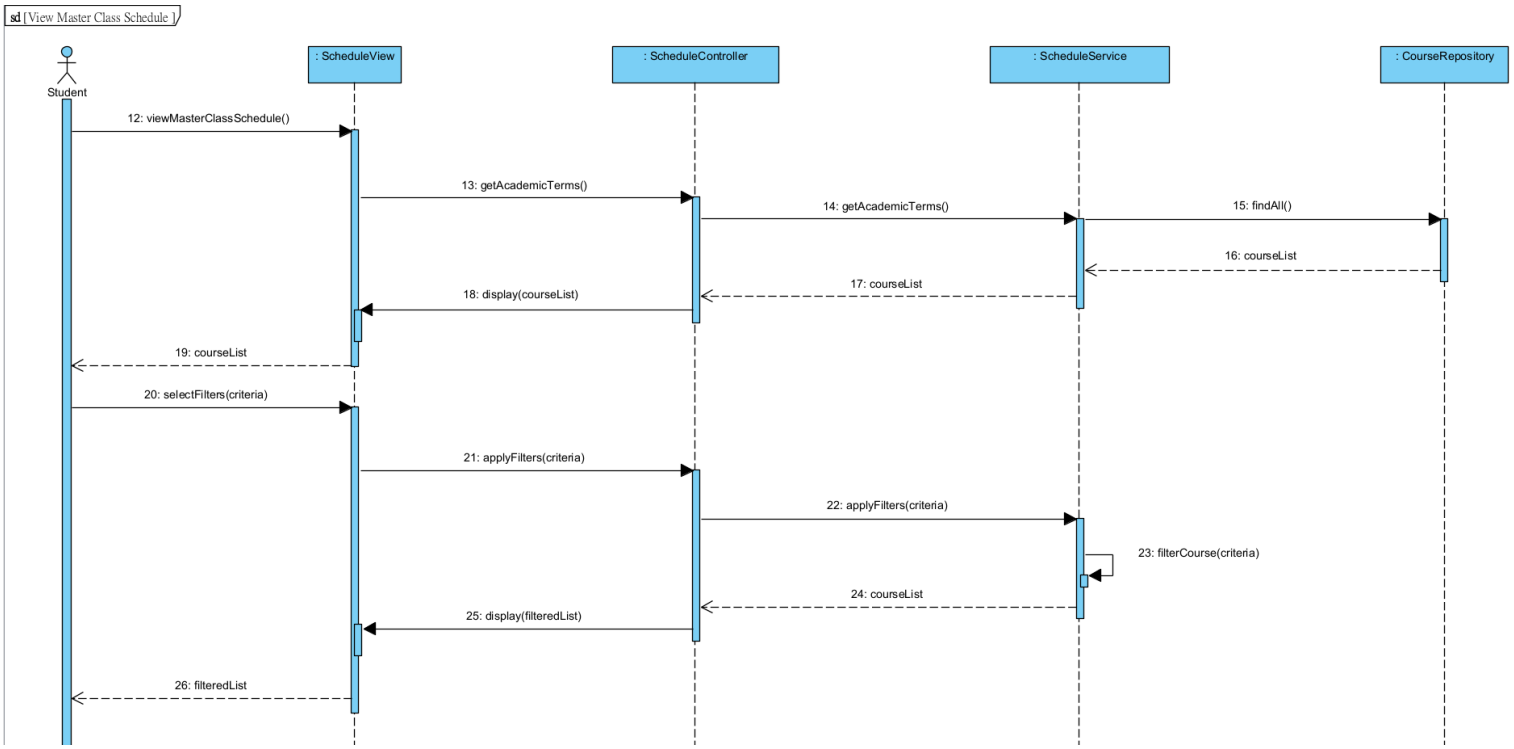
Use Case Name:	Modify User	
Actor(s):	Admin	
Description:	This use case describes the process of an admin modifying the details of a user.	
Typical Course of Events:	**Actor Action** 1. Admin requests to modify a user. 3. Admin modifies the details of the user. 6. Admin requests to save the user details.	**System Response** 2. System presents the user details. 4. System validates the changes. 5. System presents the updated user details. 7. System saves the modified user.
Alternate Courses of events:	4a. If a change is invalid, the system presents an error message to notify the admin and prompts the admin to perform alternative actions. Rejoin Step 3. a. During step 3 to 6, If the admin requests to stop modifying and has made changes to the user details, the system prompts the admin to save the changes. If the admin chooses to save, rejoin step 4. If the admin confirms to leave, the use case ends.	
Precondition:	The admin is logged in.	
Postcondition:	The system has modified and saved the user details as requested.	

Sequence diagram

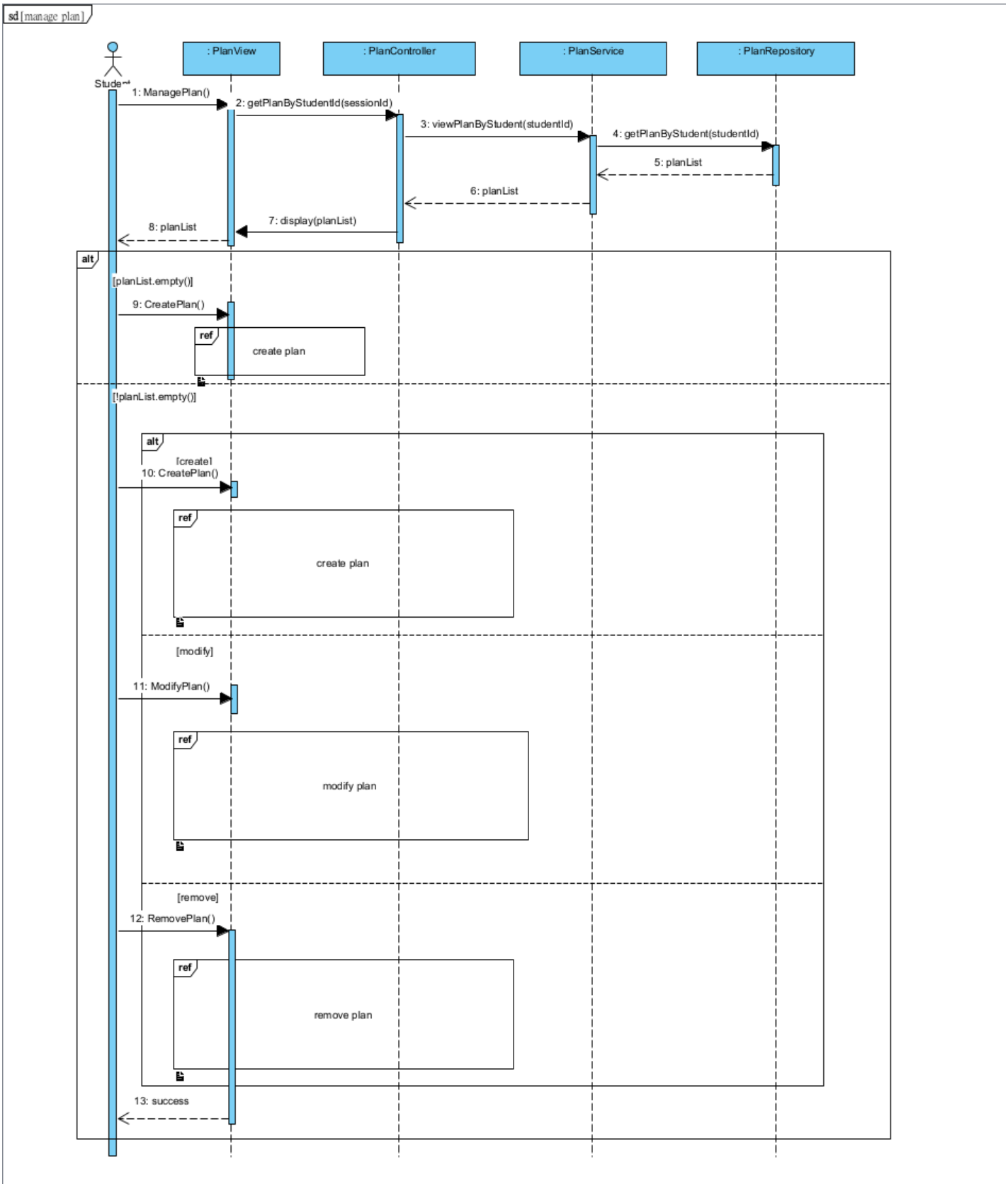
Login



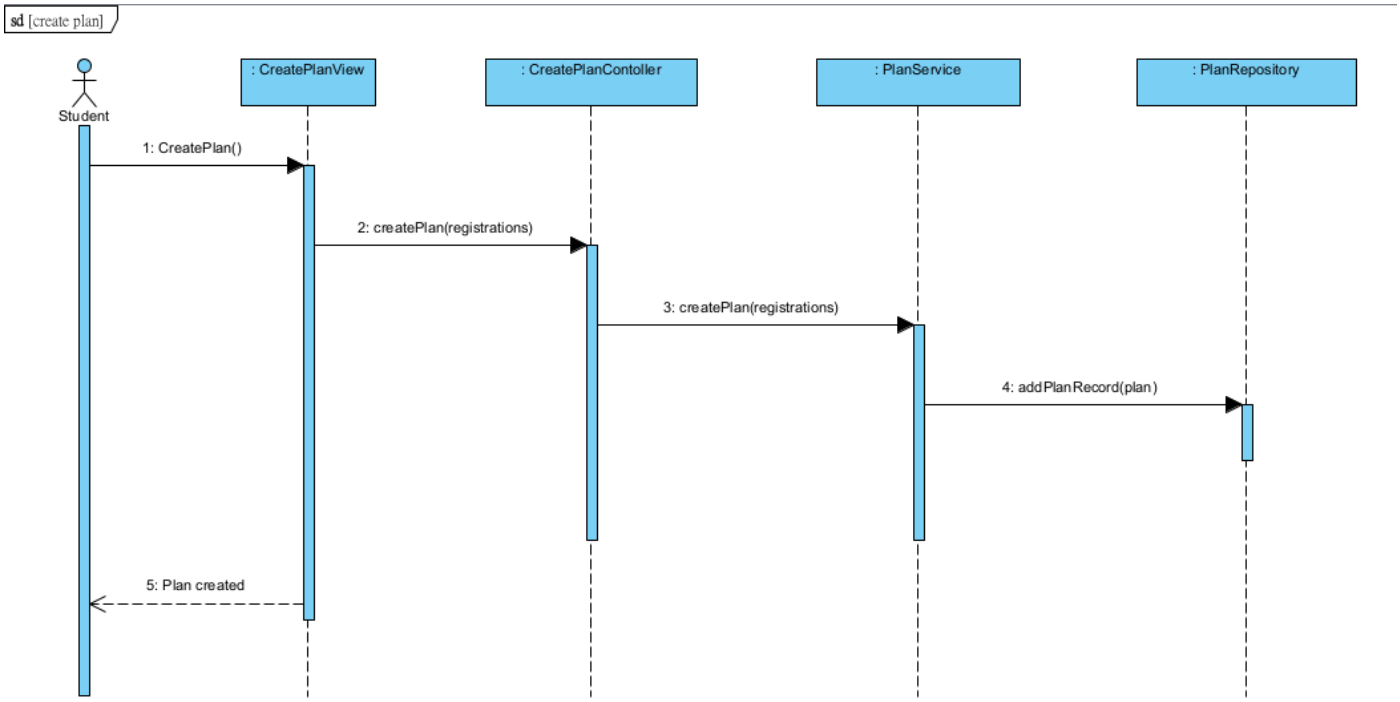
View master class schedule



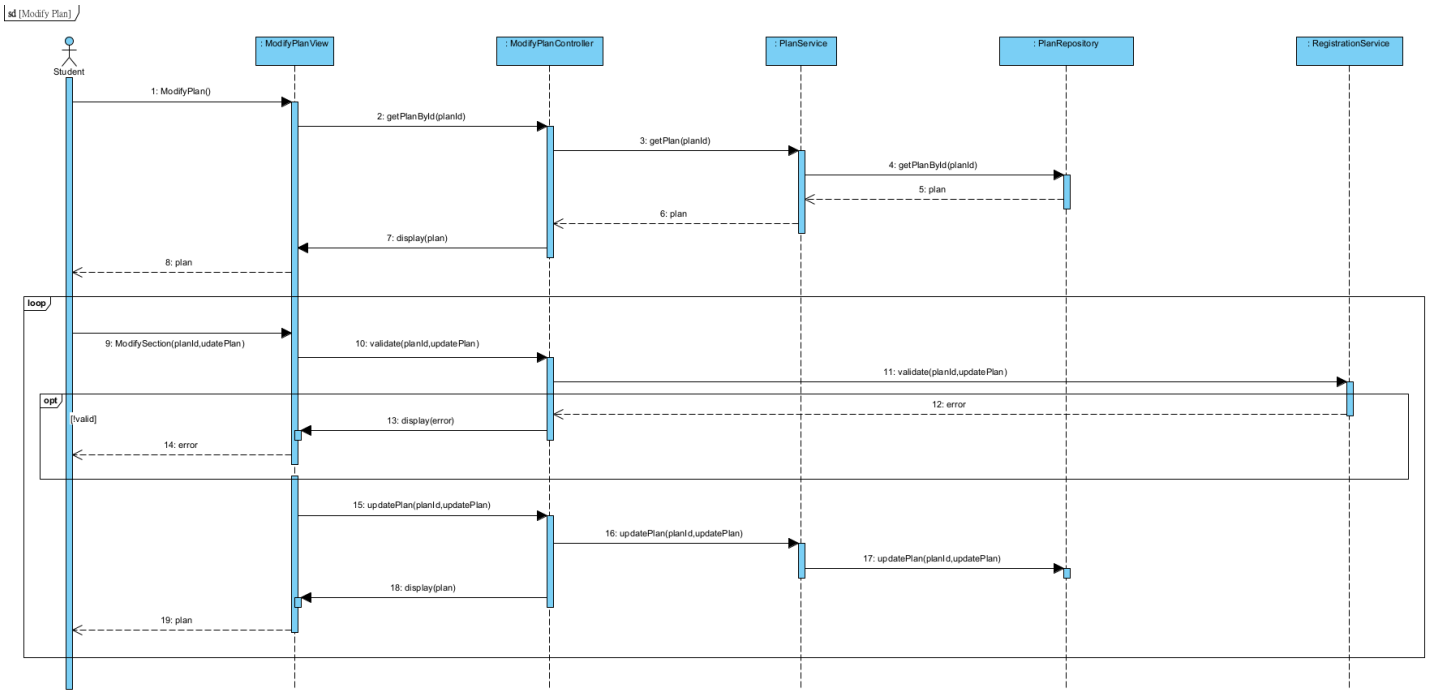
Manage Plan


sd [manage plan]
: PlanView
: PlanController
: PlanService
: PlanRepository
1: ManagePlan()
2: getPlanByStudentId(sessionId)
3: viewPlanByStudent(studentId)
4: getPlanByStudent(studentId)
5: planList
6: planList
7: display(planList)
8: planList
alt
[planList.empty()]
9: CreatePlan()
ref
create plan
[!planList.empty()]
alt
[create]
10: CreatePlan()
ref
create plan
[modify]
11: ModifyPlan()
ref
modify plan
[remove]
12: RemovePlan()
ref
remove plan
13: success

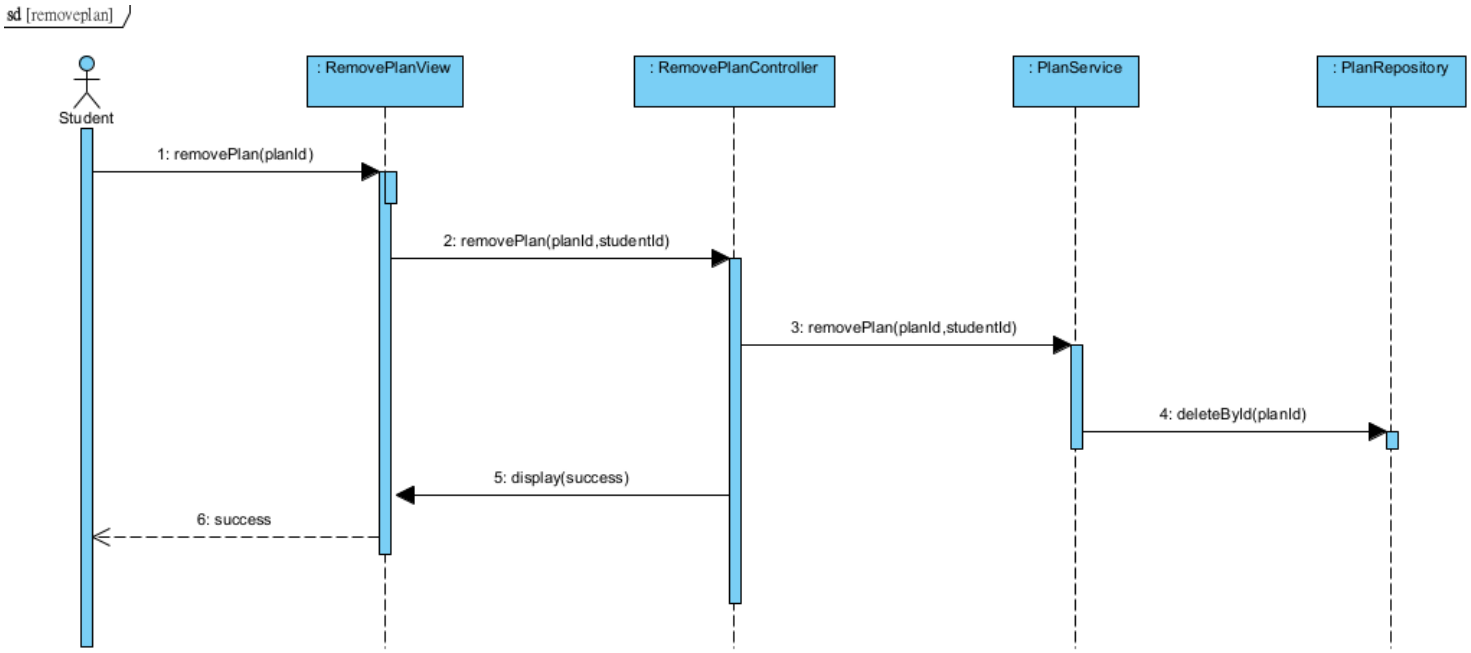
Create Plan


sd [create plan]
Student
: CreatePlanView
: CreatePlanContoller
: PlanService
: PlanRepository
1: CreatePlan()
2: createPlan(registrations)
3: createPlan(registrations)
4: addPlanRecord(plan)
5: Plan created

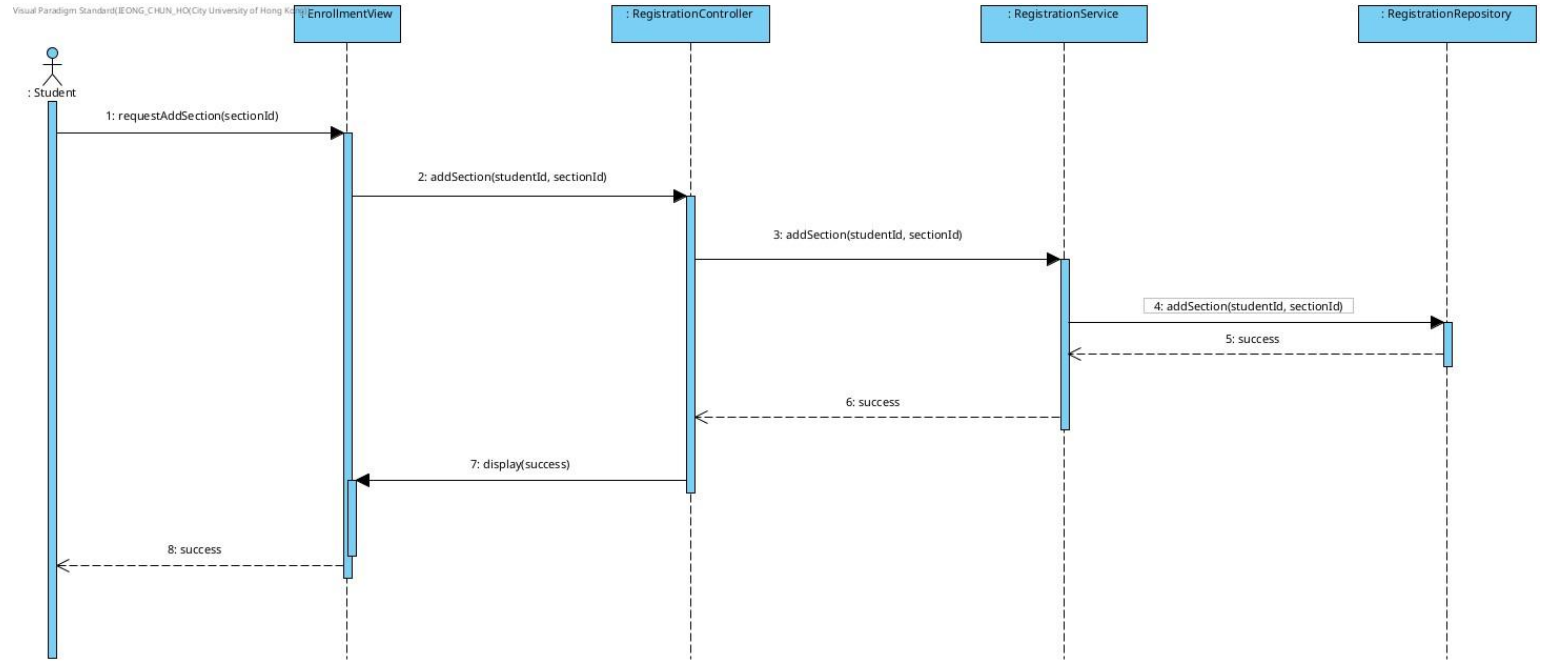
Modify Plan


sd [Modify Plan]
Student
: ModifyPlanView
: ModifyPlanController
: PlanService
: PlanRepository
: RegistrationService
1: ModifyPlan()
2: getPlanById(planId)
3: getPlan(planId)
4: getPlanById(planId)
5: plan
6: plan
7: display(plan)
8: plan
loop
9: ModifySection(planId,udatePlan)
10: validate(planId,updatePlan)
11: validate(planId,updatePlan)
12: error
opt
[!valid]
13: display(error)
14: error
15: updatePlan(planId,updatePlan)
16: updatePlan(planId,updatePlan)
17: updatePlan(planId,updatePlan)
18: display(plan)
19: plan

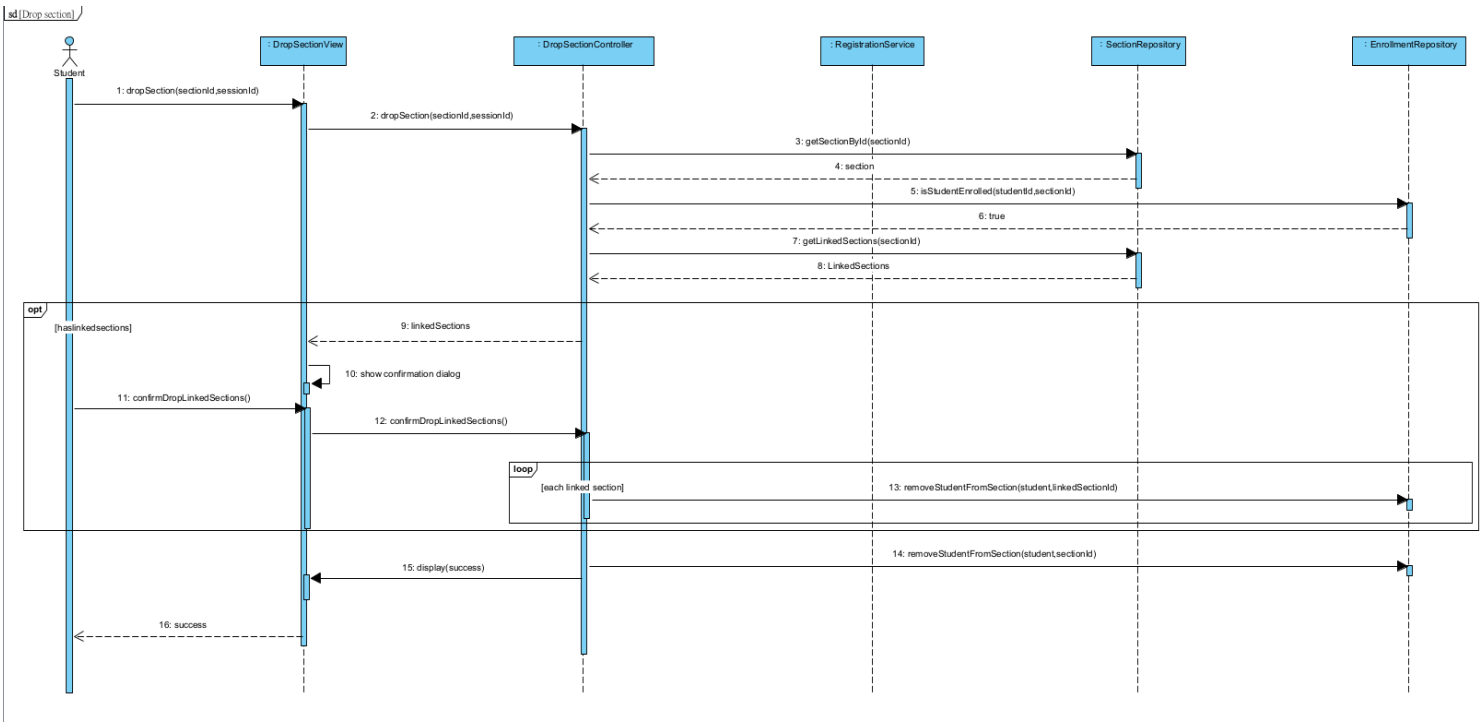
Remove Plan


sd [removeplan]
Student
: RemovePlanView
: RemovePlanController
: PlanService
: PlanRepository
1: removePlan(planId)
2: removePlan(planId,studentId)
3: removePlan(planId,studentId)
4: deleteById(planId)
5: display(success)
6: success

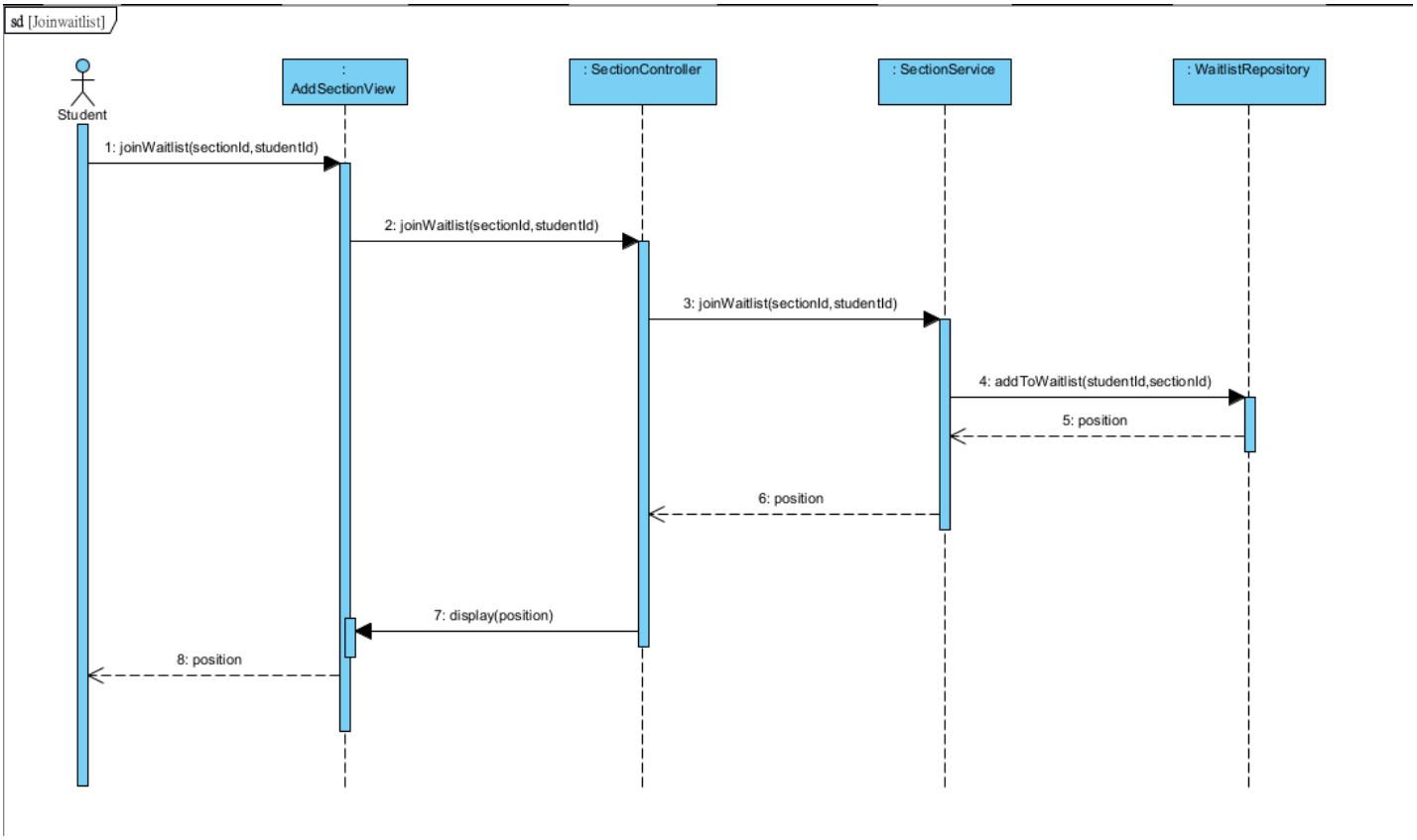
Add Section


: EnrollmentView
: RegistrationController
: RegistrationService
: RegistrationRepository
: Student
1: requestAddSection(sectionId)
2: addSection(studentId, sectionId)
3: addSection(studentId, sectionId)
4: addSection(studentId, sectionId)
5: success
6: success
7: display(success)
8: success

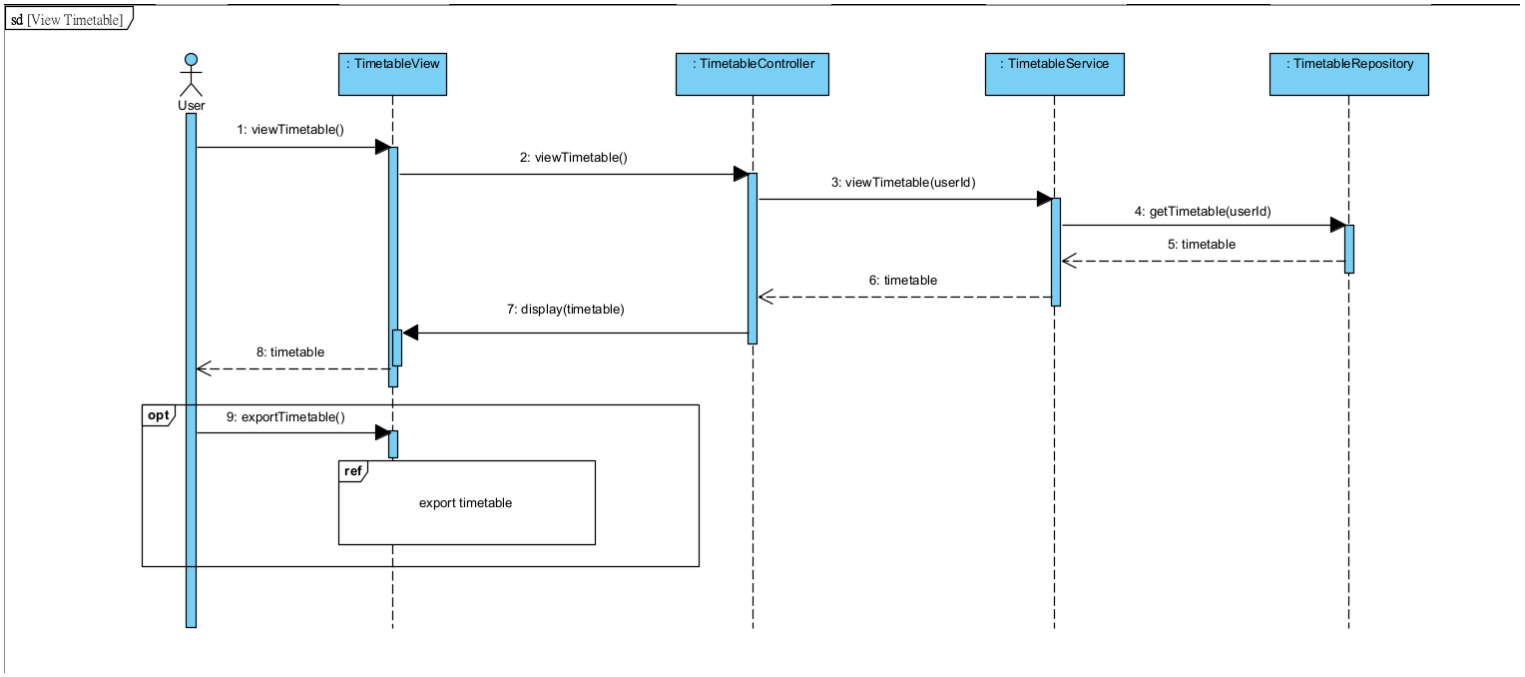
Drop section



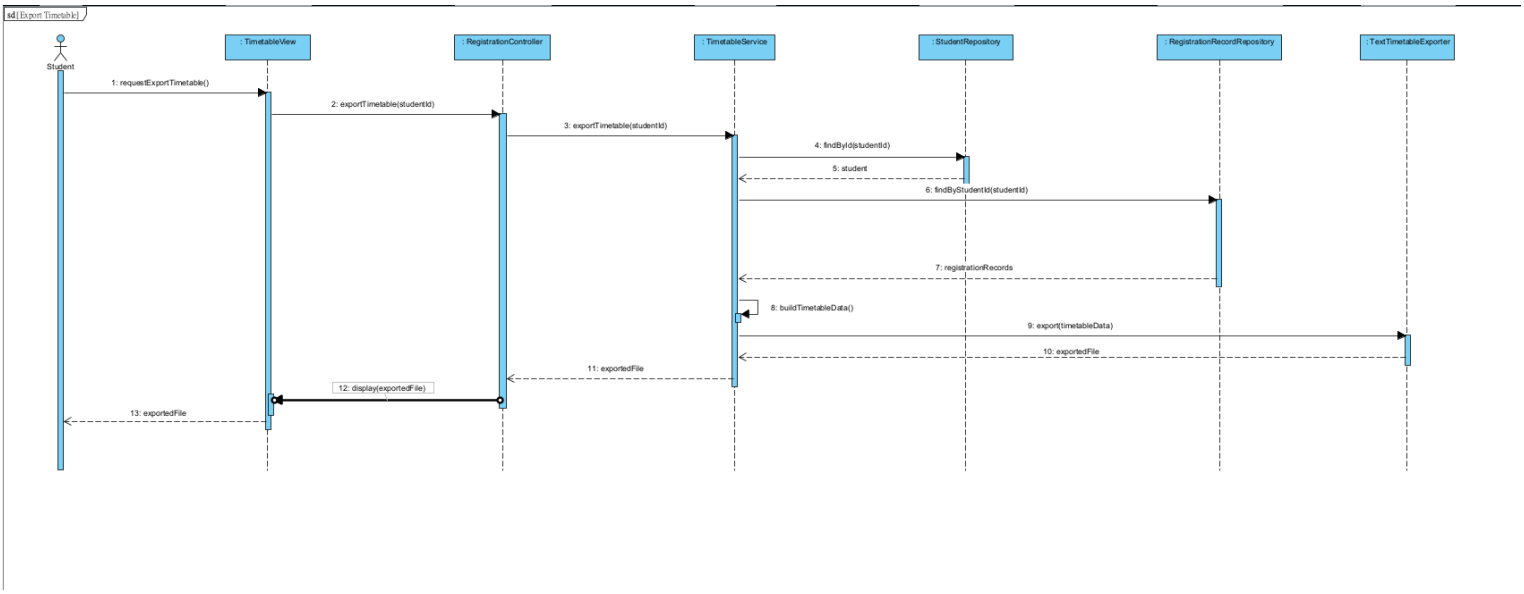
Drop Waitlist



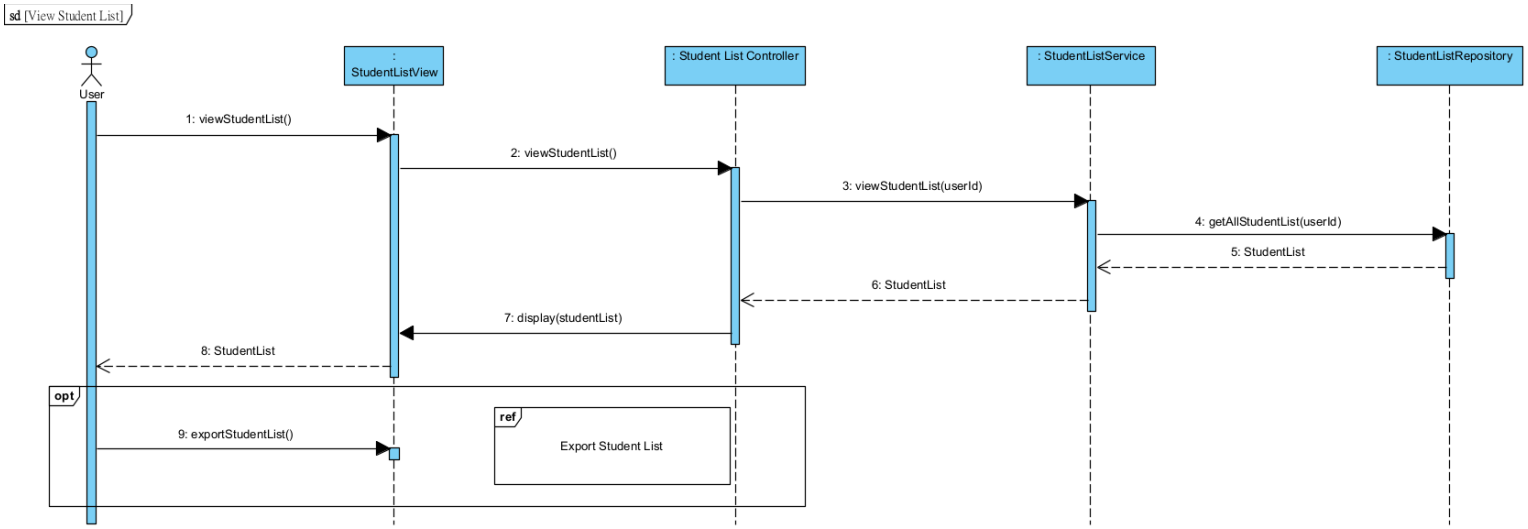
Check Timetable



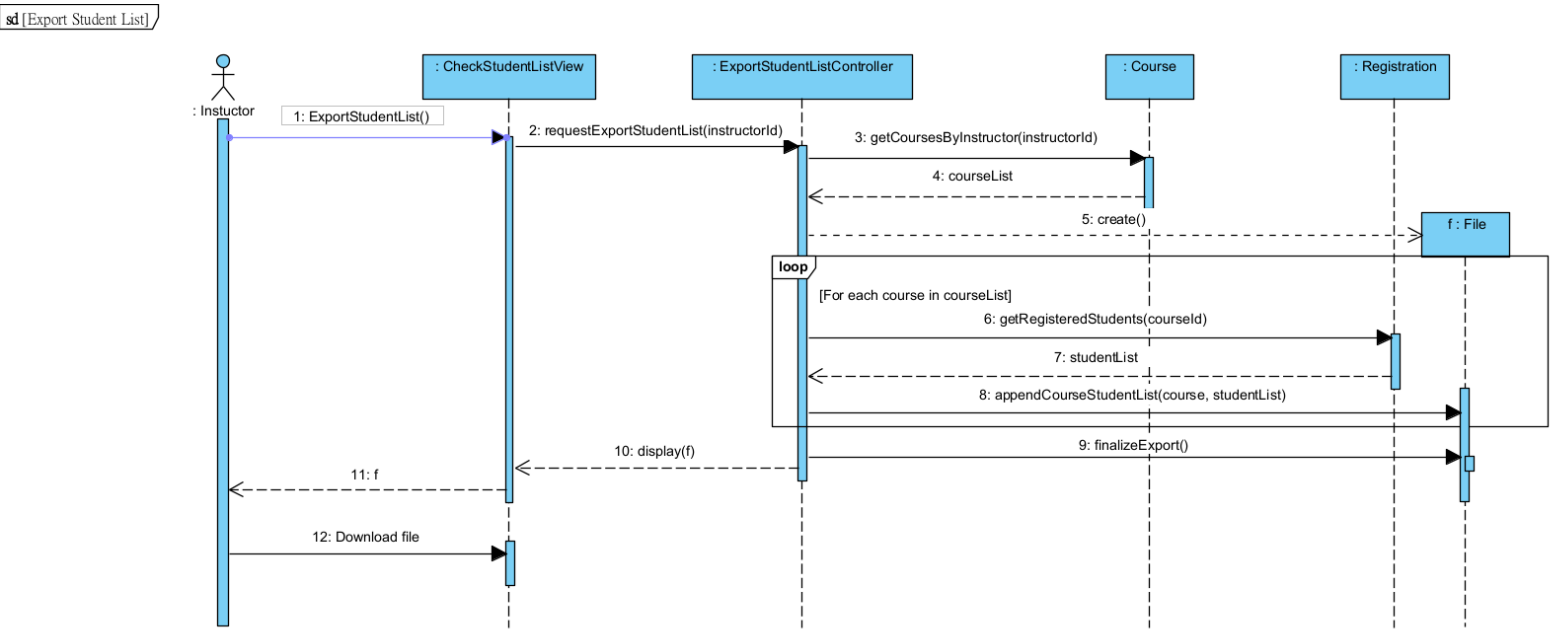
Export Timetable



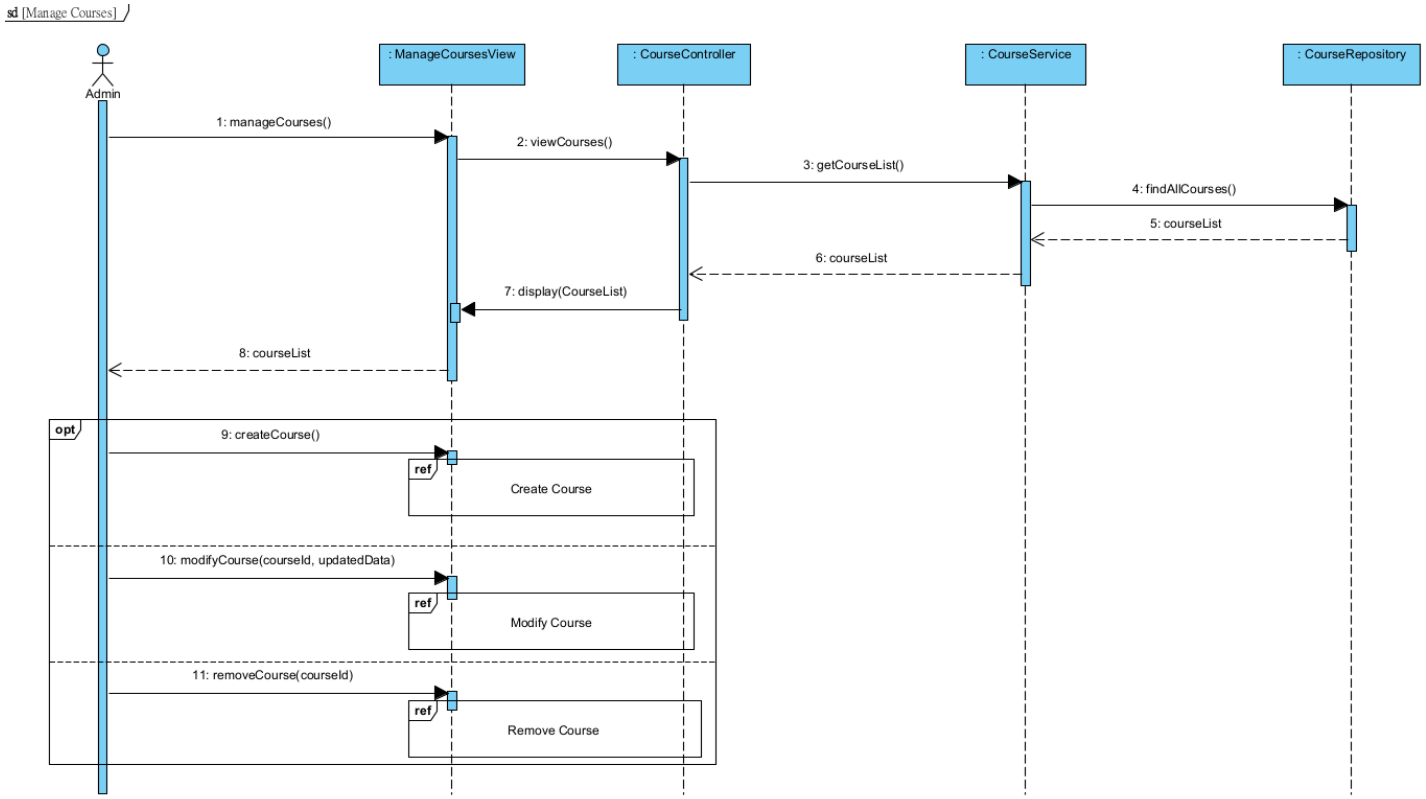
Check Student List



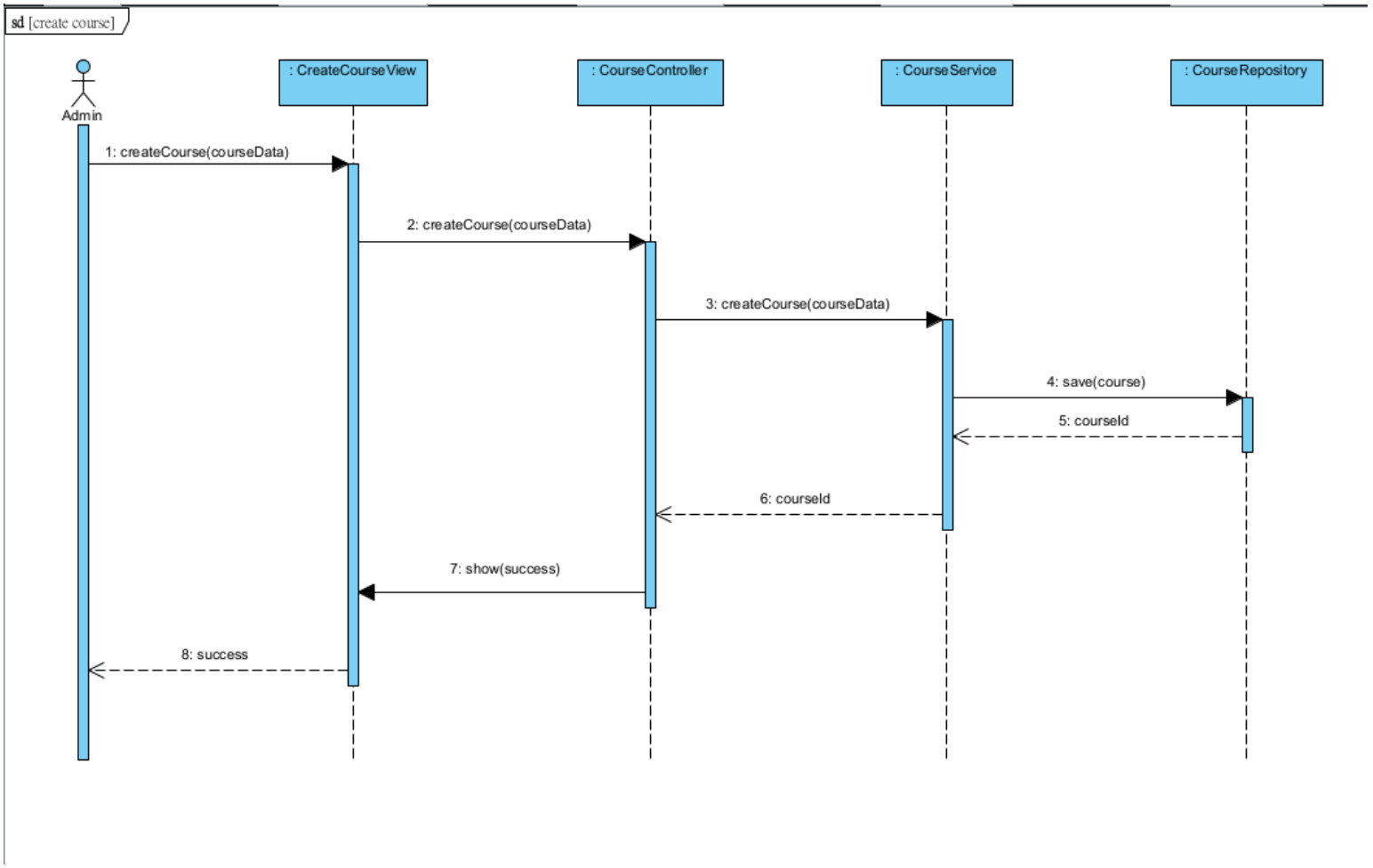
Export Student List



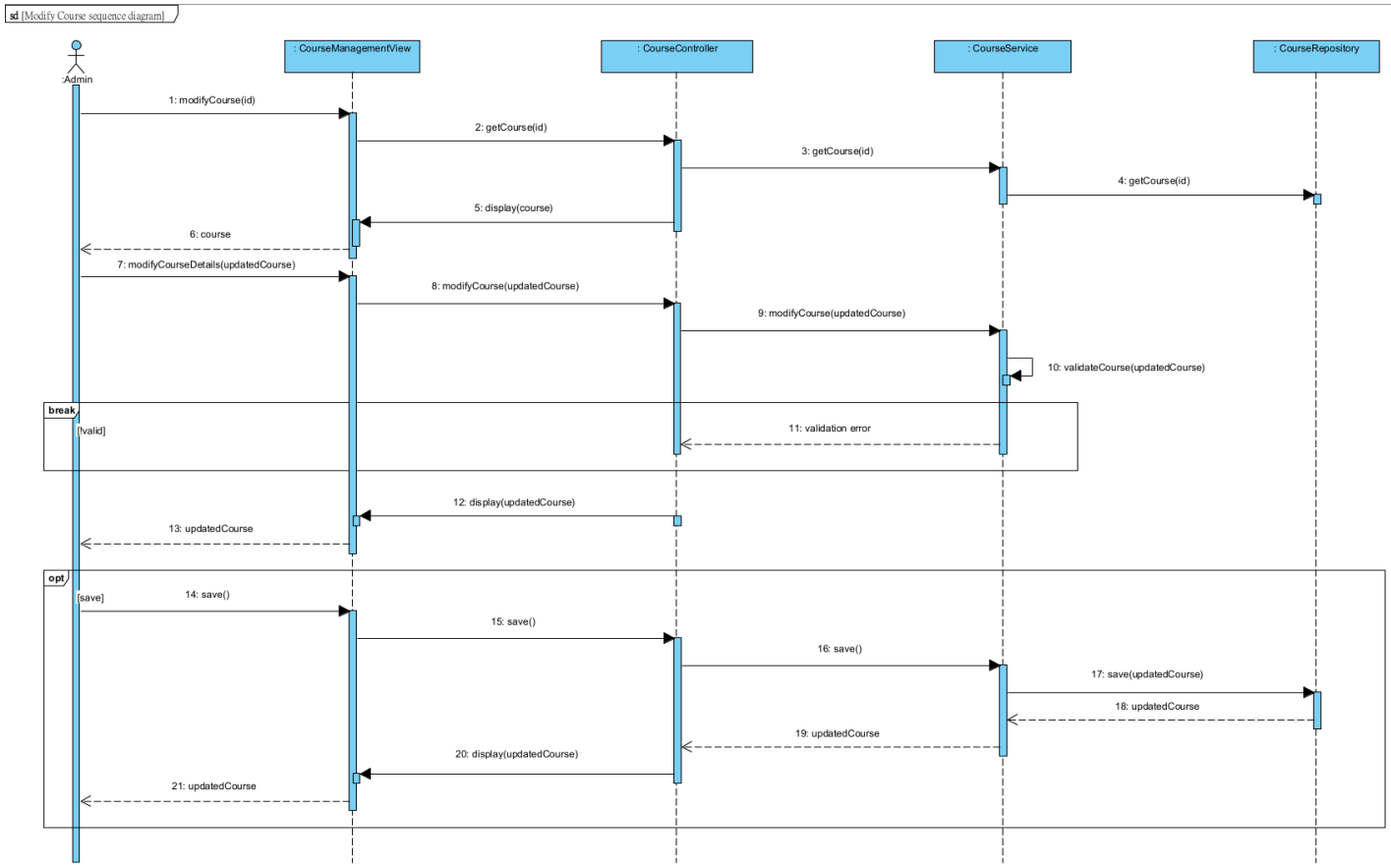
Manage Course



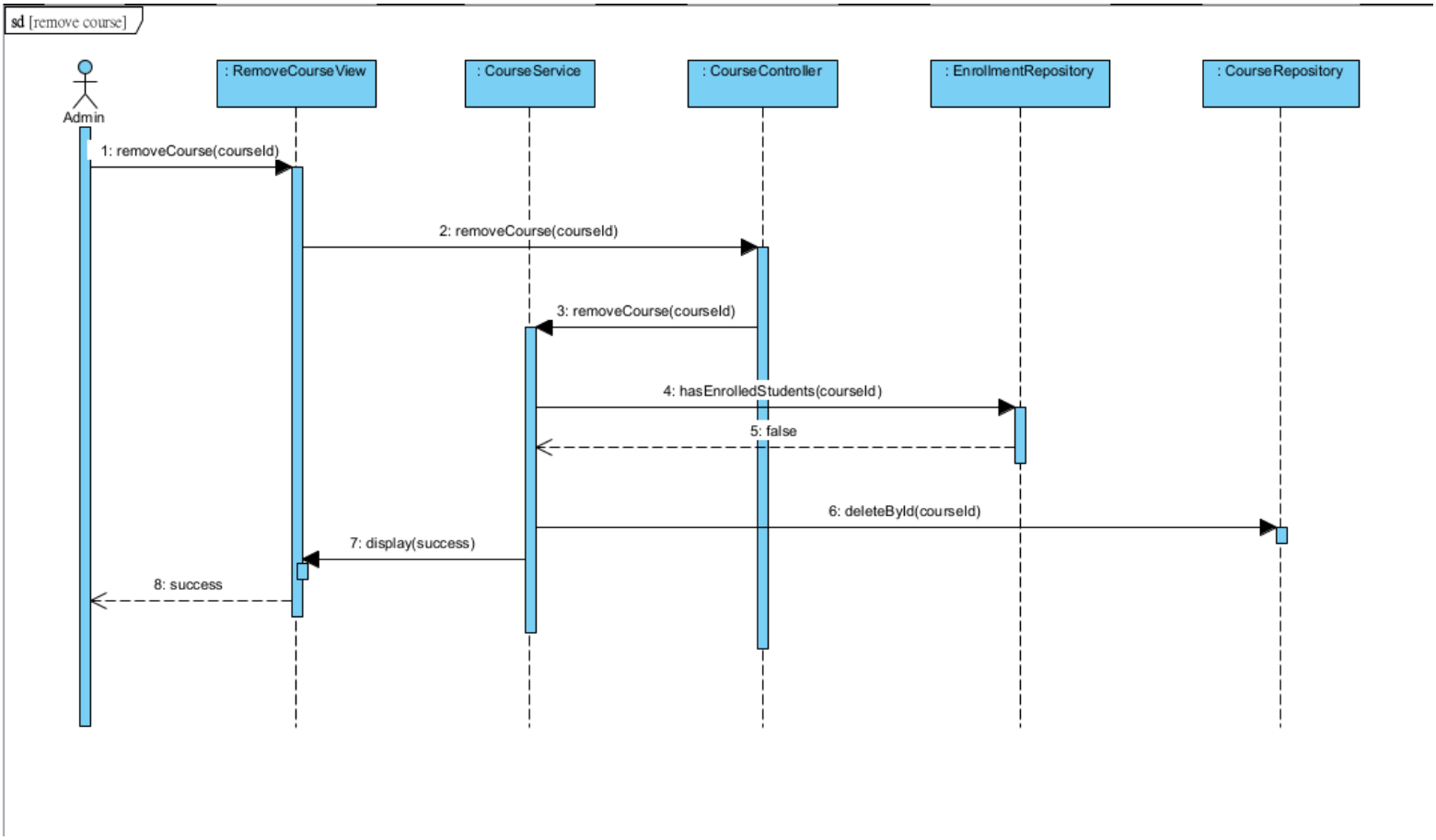
Create Course



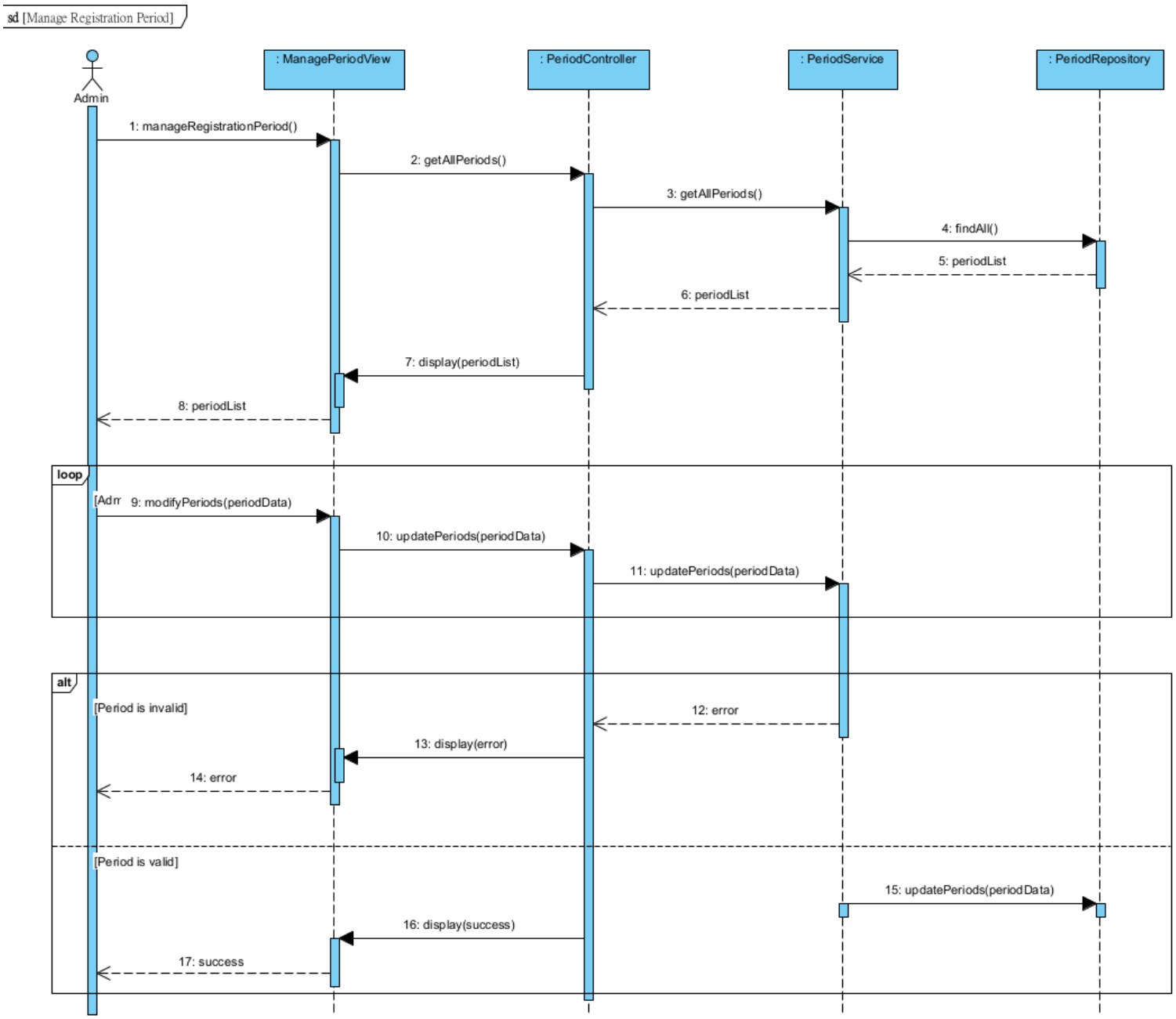
Modify Course



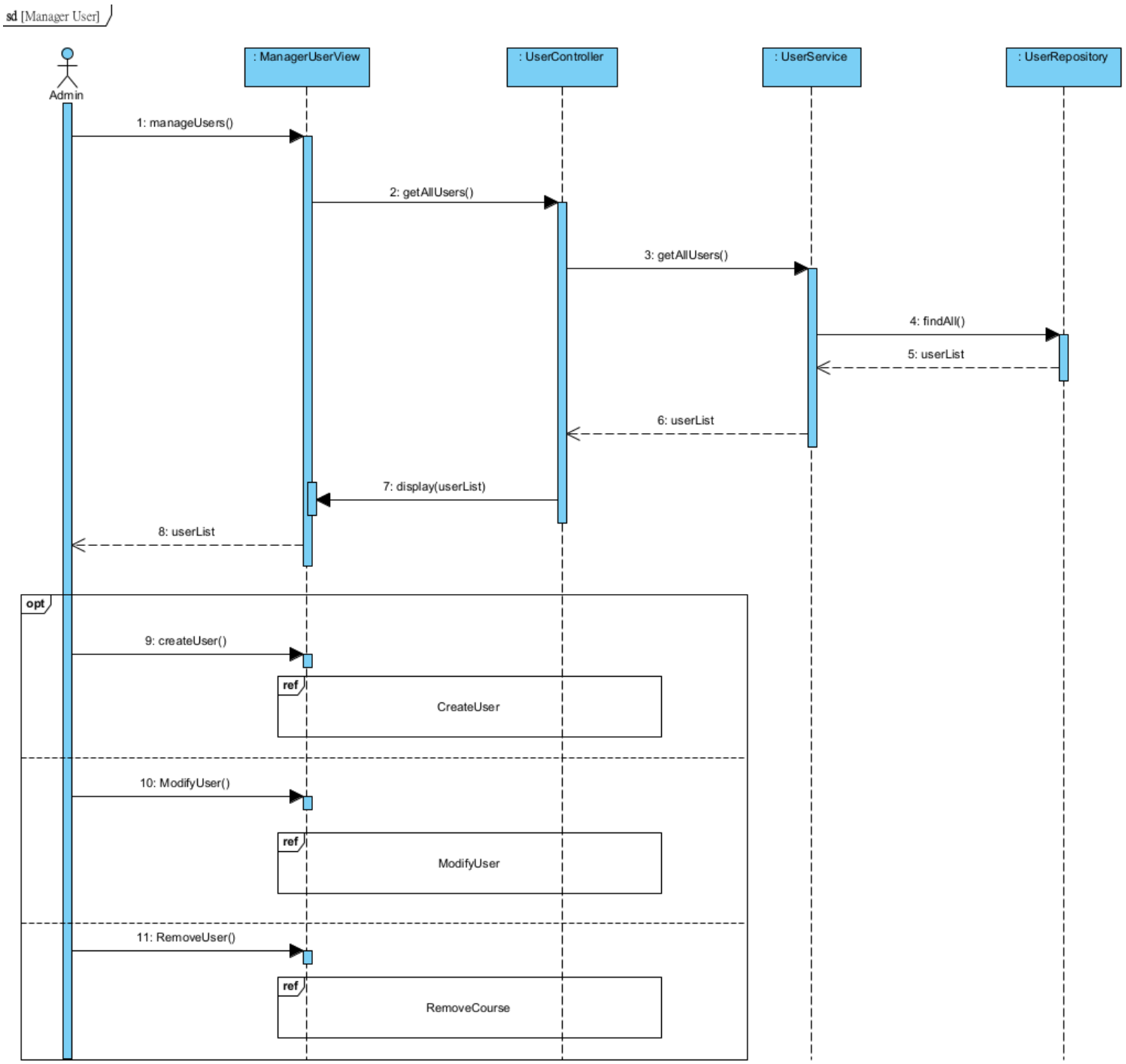
Remove Course



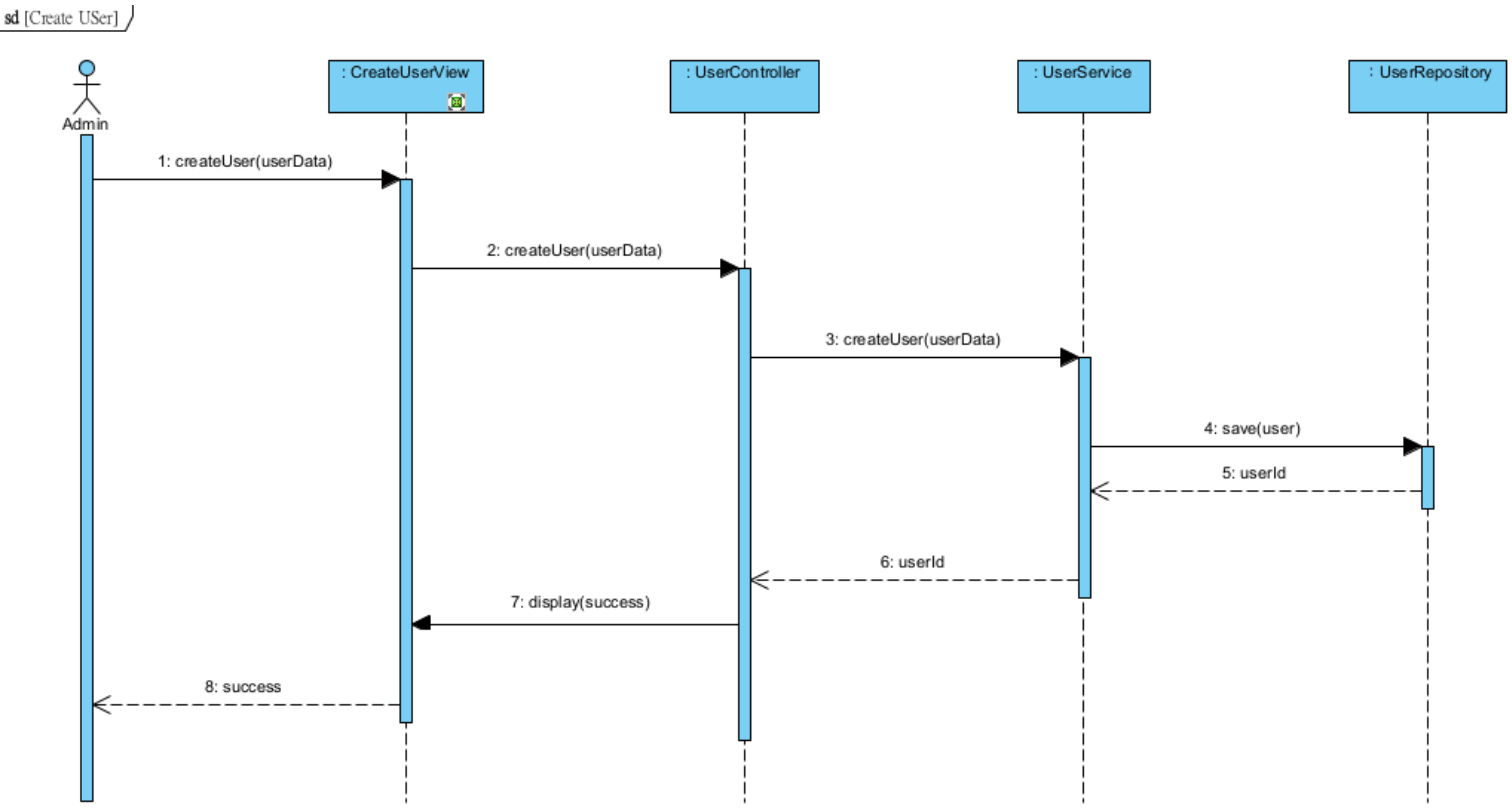
Manage Registration Period



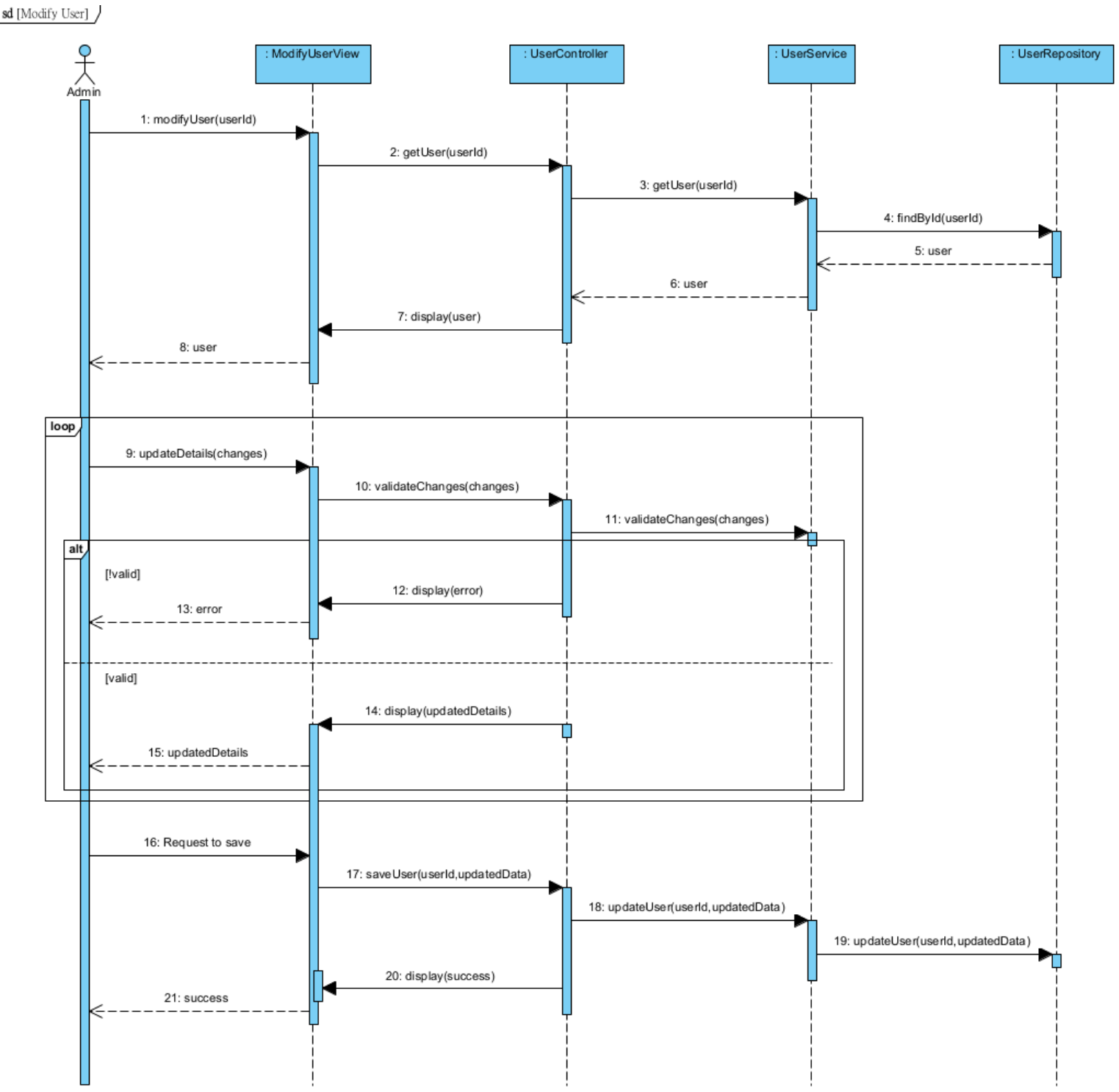
Manage User


sd [Manager User]
Admin
: ManagerUserView
: UserController
: UserService
: UserRepository
1: manageUsers()
2: getAllUsers()
3: getAllUsers()
4: findAll()
5: userList
6: userList
7: display(userList)
8: userList
opt
9: createUser()
ref
CreateUser
10: ModifyUser()
ref
ModifyUser
11: RemoveUser()
ref
RemoveCourse

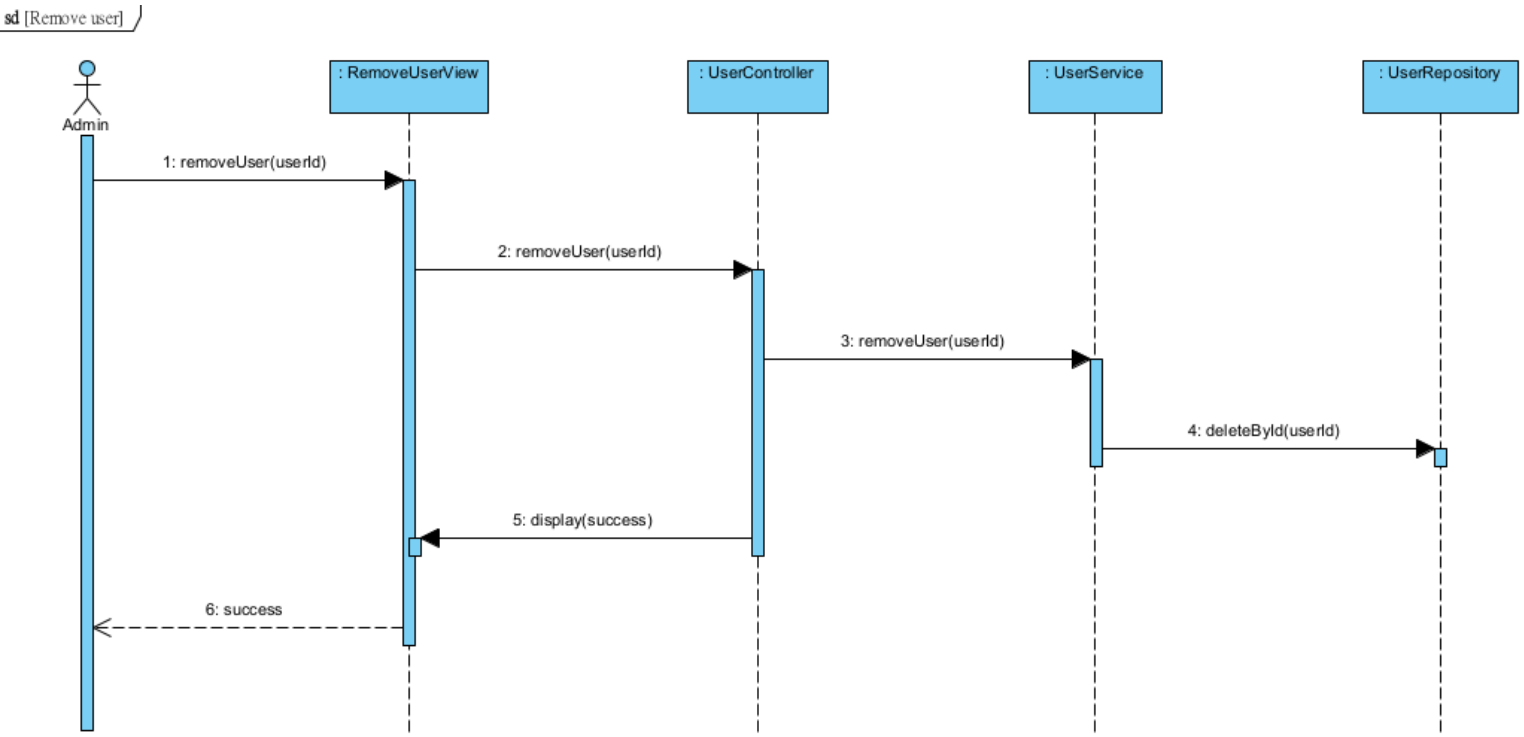
Create User


sd [Create USer]
Admin
: CreateUserView
: UserController
: UserService
: UserRepository
1: createUser(userData)
2: createUser(userData)
3: createUser(userData)
4: save(user)
5: userId
6: userId
7: display(success)
8: success

Modify User


sd [Modify User]
Admin
: ModifyUserView
: UserController
: UserService
: UserRepository
1: modifyUser(userId)
2: getUser(userId)
3: getUser(userId)
4: findById(userId)
5: user
6: user
7: display(user)
8: user
loop
9: updateDetails(changes)
10: validateChanges(changes)
11: validateChanges(changes)
alt
[!valid]
12: display(error)
13: error
[valid]
14: display(updatedDetails)
15: updatedDetails
16: Request to save
17: saveUser(userId,updatedData)
18: updateUser(userId, updatedData)
19: updateUser(userId, updatedData)
20: display(success)
21: success

Remove User


sd [Remove user]
Admin
: RemoveUserView
: UserController
: UserService
: UserRepository
1: removeUser(userId)
2: removeUser(userId)
3: removeUser(userId)
4: deleteById(userId)
5: display(success)
6: success

Implementation

✅ Fully Implemented

⚠️ Partially Implemented

❌ Not Implemented

Use Case	Status	Human-refined
Login	✅	Yes
View Master Class Schedule	✅	Yes
Manage Plans	⚠️	Yes
Create Plan	⚠️	Yes
Modify Plan	⚠️	Yes
Remove Plan	⚠️	Yes
Add Section	✅	Yes
Drop Section	✅	Yes
Join Waitlist	✅	Yes
Drop Waitlist	✅	Yes
View Timetable	✅	Yes
Export Timetable	✅	Yes
View Student List	❌	No
Export Student List	✅	Yes
Manage Course	✅	Yes
Create Course	✅	Yes

Use Case	Status	Human-refined
Modify Course	✅	Yes
Remove Course	✅	Yes
Manage Registration Period	✅	Yes
Manage User	✅	Yes
Create User	✅	Yes
Modify User	✅	Yes
Remove User	✅	Yes

System Prototype

Command Line Interface showcase

Startup and login:

```
crs> login admin admin123
Logged in as ADMIN: admin

crs> whoami
admin (ADMIN)
Admin{staffId=1, userEID='admin', userName='Admin'}
```

Listing courses and master schedule:

```
crs> list-courses
CS1010 | Introduction to Programming | credits=3 | sections=2
CS3342 | Software Design | credits=3 | sections=1

crs> view-master-schedule
========== MASTER CLASS SCHEDULE ==========

COURSE: CS3342 - Software Design
Credits: 3
  Sections:
    ----------------------------------------
    Section ID: 1
    Type: LECTURE
    Time: Mon 09:00 to Mon 10:50
    Venue: Y4701
    Instructors: Umiri Yahata
    Enrollment: 12/80 (Available: 68)
    Waitlist: 0/20 (Available: 20)
==========================================
```

Student registration:

```
crs> login student1 student123
Logged in as STUDENT: student1

crs> add-section 1
Registration added.

crs> show-timetable
STUDENT TIMETABLE
Student ID: 1
Generated At: 2026-04-27 14:32

DAY    TIME          COURSE       SEC    TYPE               VENUE
------------------------------------------------------------------------
Mon    09:00-10:50   CS3342       1      LECTURE            Y4701

crs> export-timetable
Timetable exported to C:\...\student-1-timetable.txt
```

Registration plan workflow：

```
crs> create-plan 1
Created plan 3 with priority=1

crs> add-plan-entry 3 2 SELECTED true
Added plan entry 5

crs> list-plans
planId=3 | priority=1 | status=NOT_ATTEMPTED | entries=1 | summary=-
  entryId=5 | sectionId=2 | type=SELECTED | status=NOT_ATTEMPTED | joinWaitlistOnAddFailure=true | reason=-

crs> reorder-plans 3,1,2
Plans reordered.
planId=3 | priority=1
planId=1 | priority=2
planId=2 | priority=3
```

Admin section management:

```
crs> admin-create-section --course CS3342 --type TUTORIAL --enroll-capacity 30 --waitlist-capacity 10 --start 2026
Section created.

crs> admin-modify-section --section-id 12 --enroll-capacity 40
Section updated.

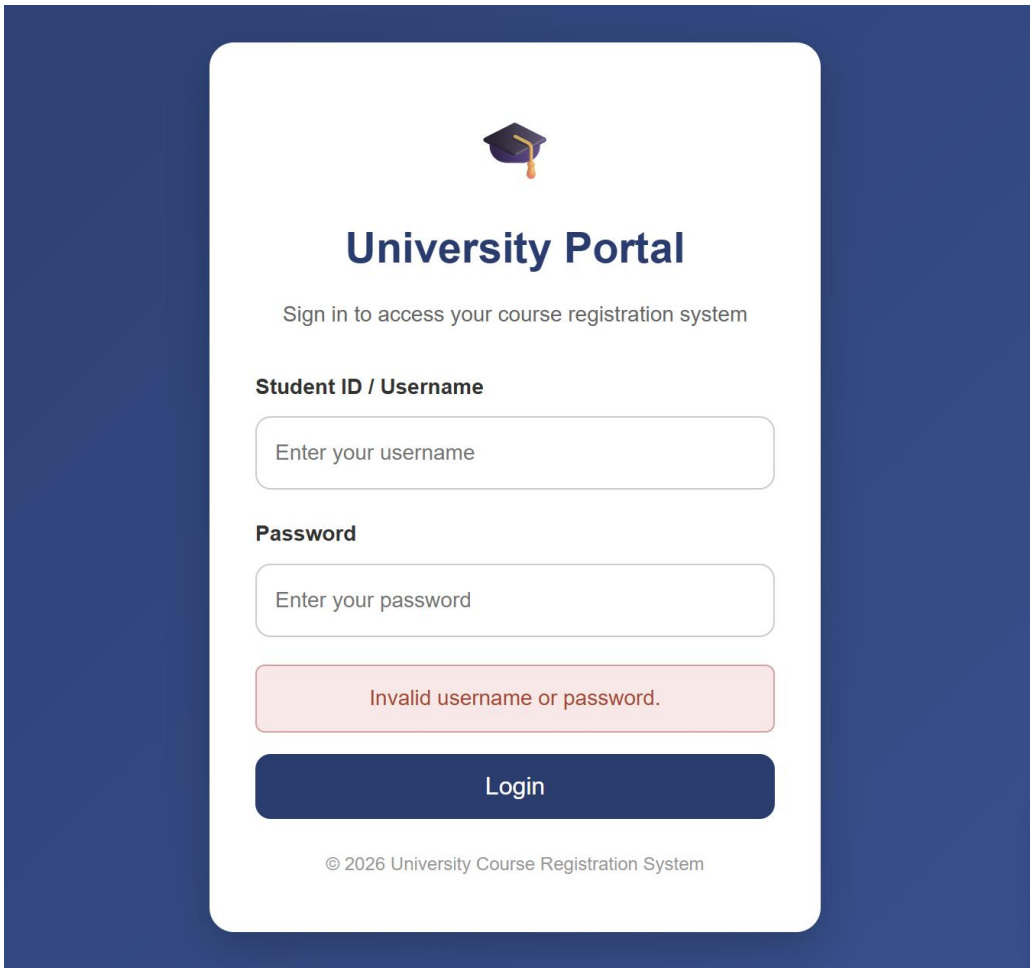
crs> admin-create-period --cohort 2024 --start 2026-05-01T09:00 --end 2026-05-31T23:59
Registration period created.
```

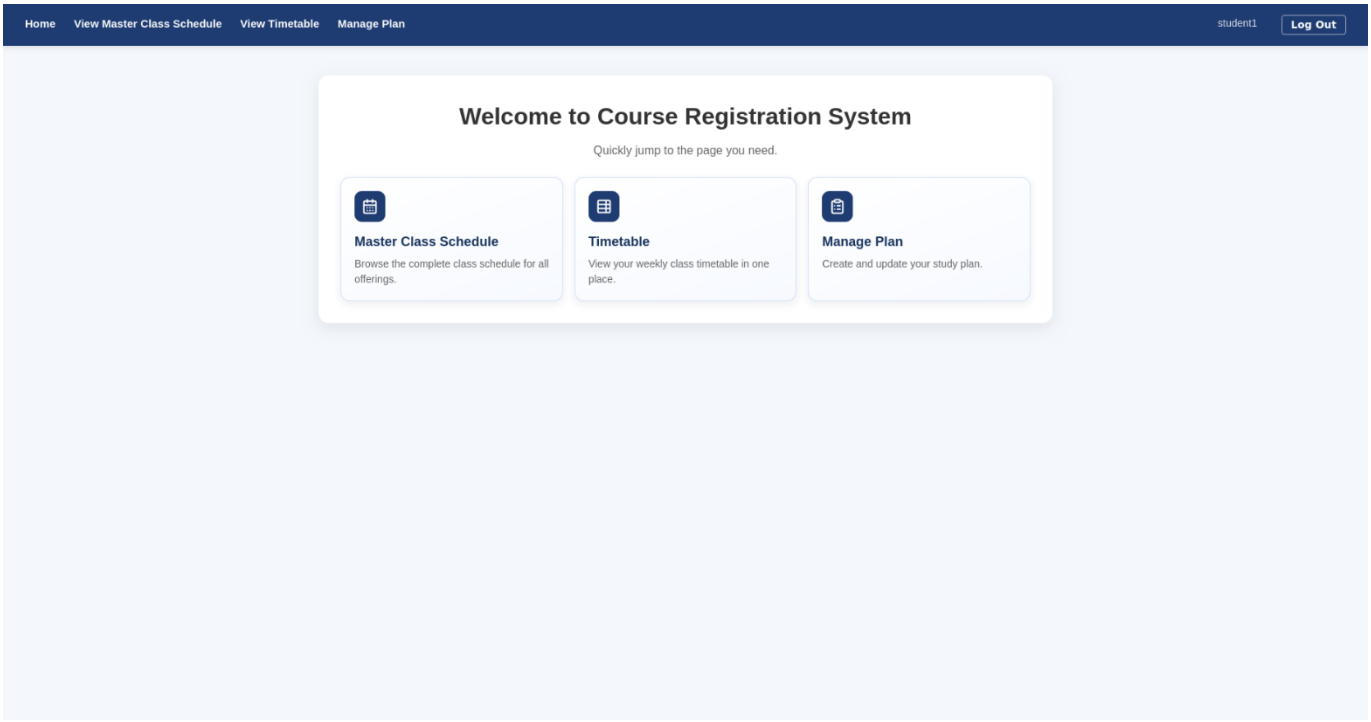
Exported timetable file format (student-0-timetable.txt):

```
STUDENT TIMETABLE
Student ID: 0
Generated At: 2026-04-27 10:15

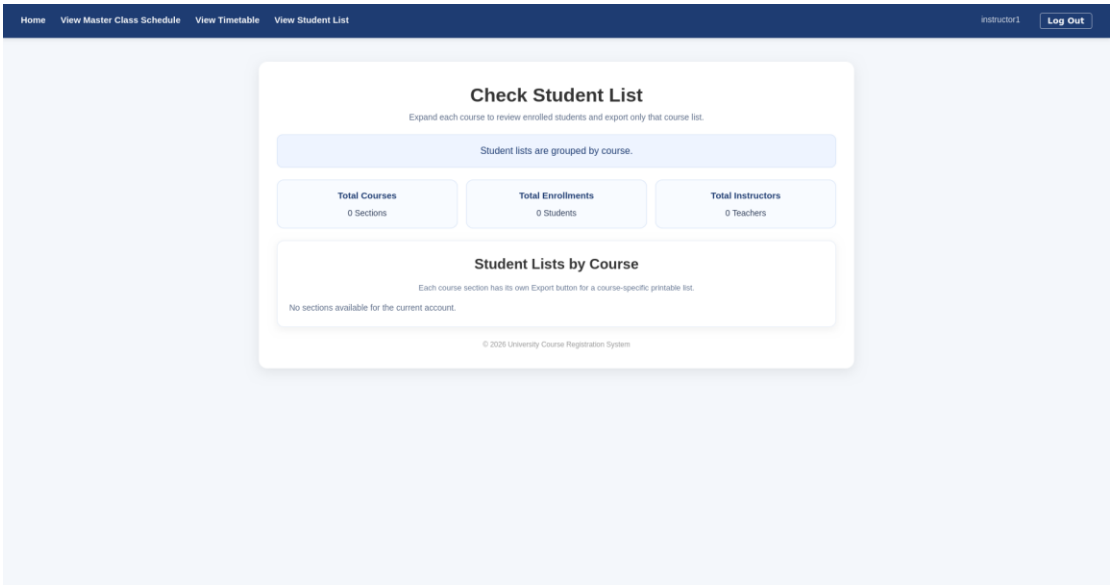
DAY    TIME          COURSE       SEC    TYPE              VENUE
------------------------------------------------------------------------
Fri    09:00-10:50   CS1010       2      LAB               Y4301
```

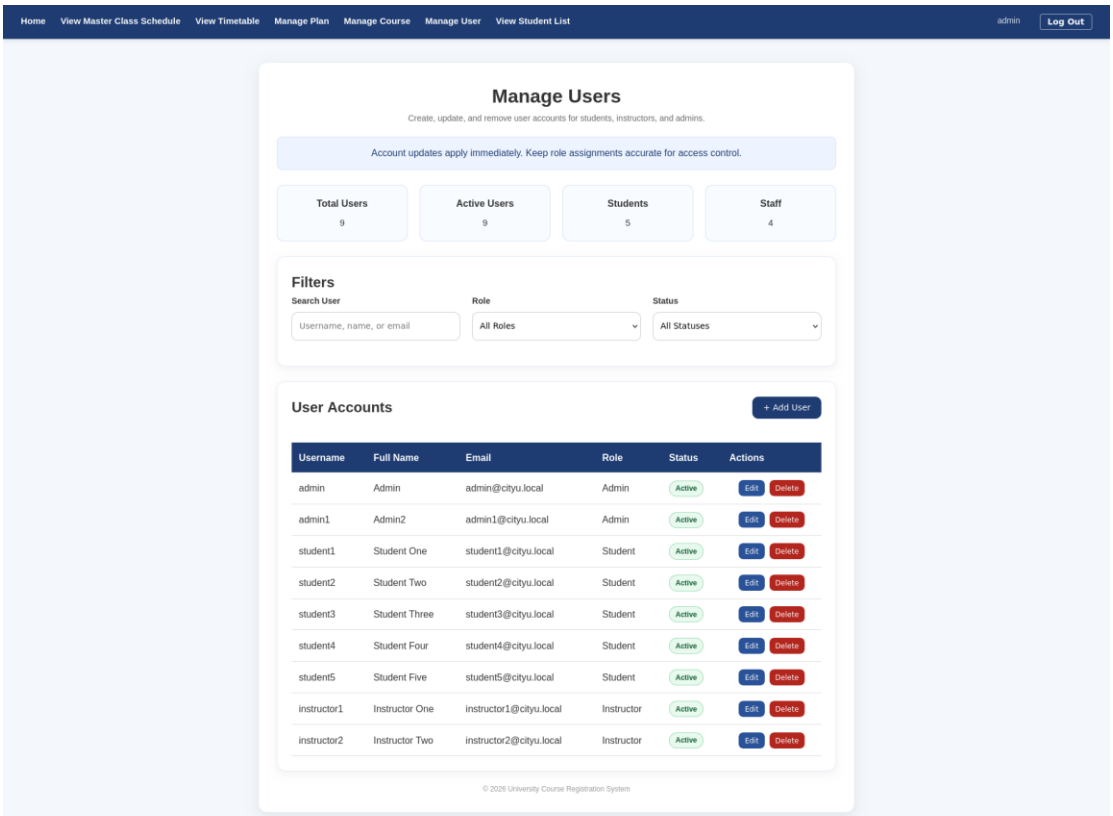
Web Interface showcase

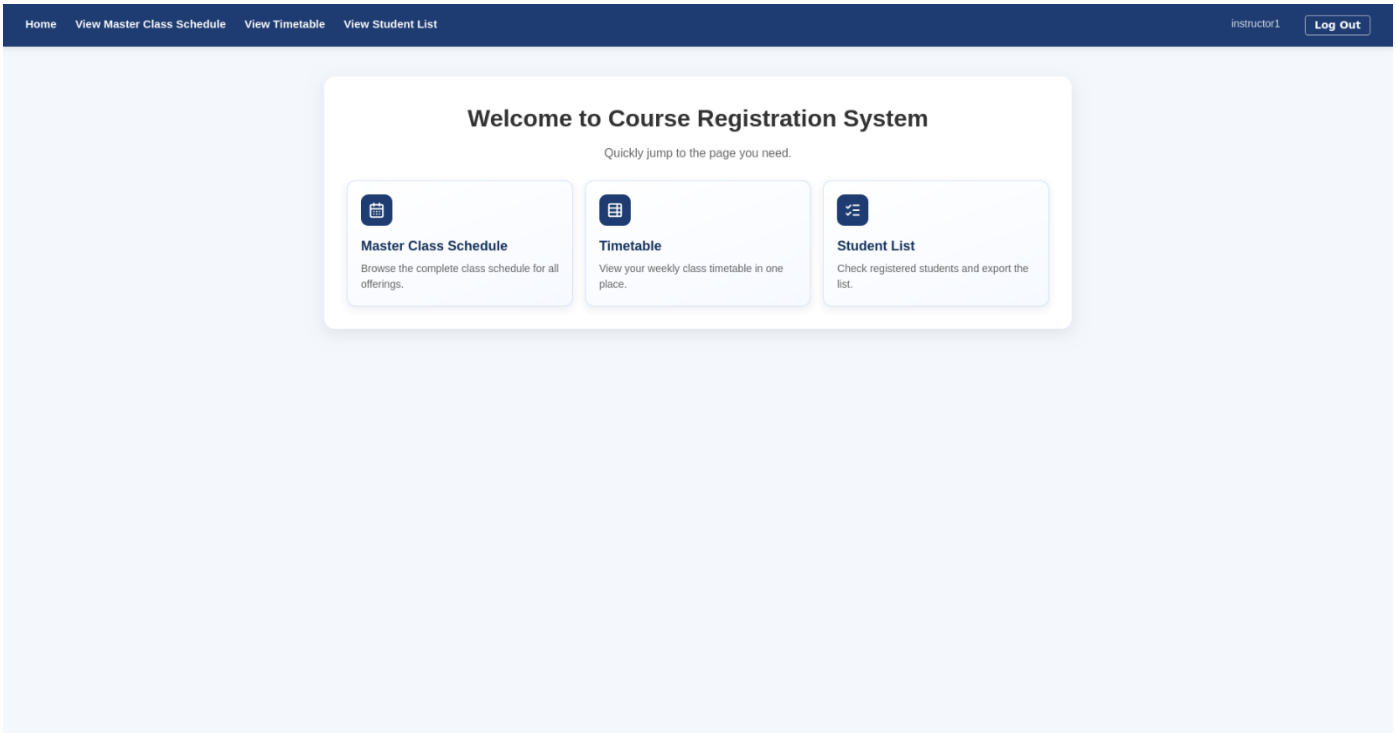


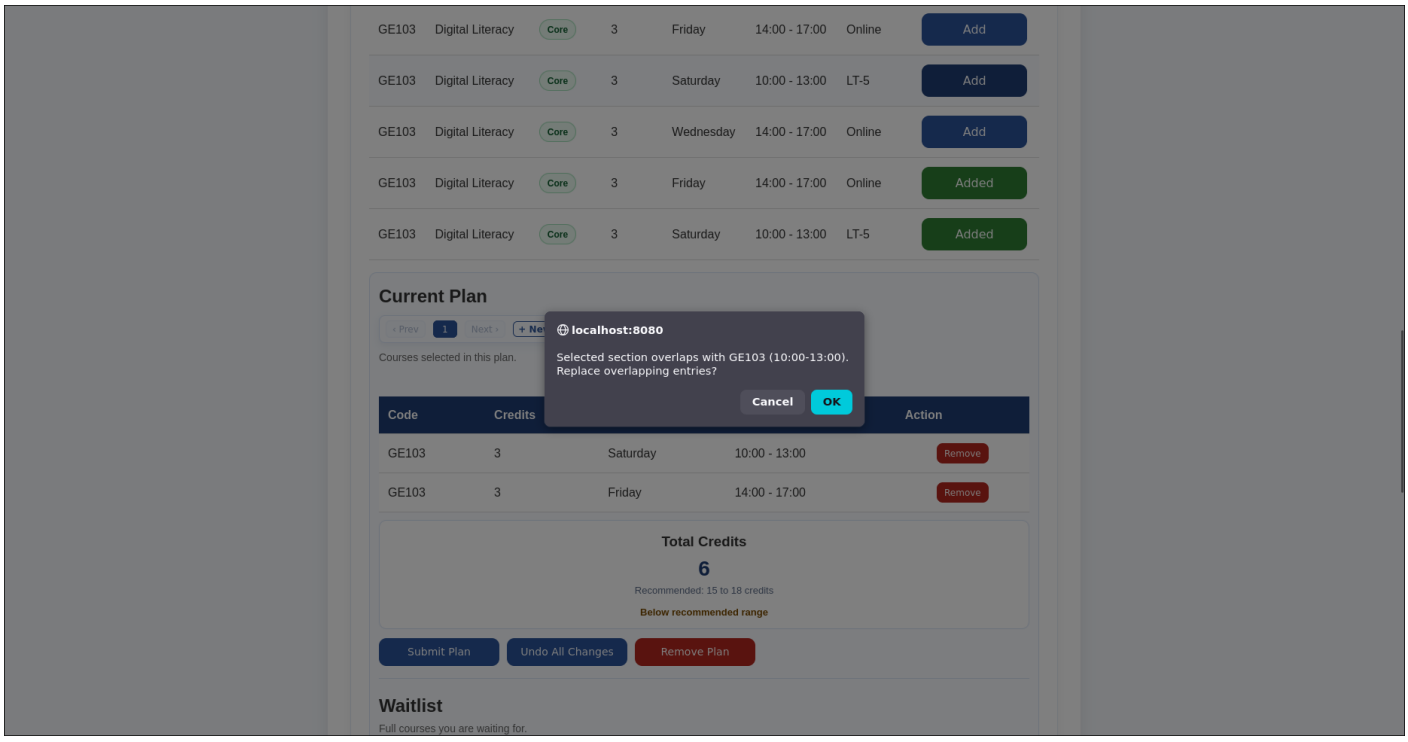

Home
View Master Class Schedule
View Timetable
Manage Plan
student1
Log Out
Welcome to Course Registration System
Quickly jump to the page you need.
Master Class Schedule
Browse the complete class schedule for all offerings.
Timetable
View your weekly class timetable in one place.
Manage Plan
Create and update your study plan.


View Master Class Schedule
Use the filters below to find matching courses.
Course Code
Add More Filters
Search Courses
Available Courses
Showing courses that match the applied filters.
Course Code
Course Title
Credits
Section
Type
Day
Time
Location
Instructor
Enrollment


Home
View Master Class Schedule
View Timetable
View Student List
instructor1
Log Out
Check Student List
Expand each course to review enrolled students and export only that course list.
Student lists are grouped by course.
Total Courses
0 Sections
Total Enrollments
0 Students
Total Instructors
0 Teachers
Student Lists by Course
Each course section has its own Export button for a course-specific printable list.
No sections available for the current account.
© 2026 University Course Registration System


Home
View Master Class Schedule
View Timetable
Manage Plan
Manage Course
Manage User
View Student List
admin
Log Out
Manage Users
Create, update, and remove user accounts for students, instructors, and admins.
Account updates apply immediately. Keep role assignments accurate for access control.
Total Users
9
Active Users
9
Students
5
Staff
4
Filters
Search User
Username, name, or email
Role
All Roles
Status
All Statuses
User Accounts
+ Add User
Username
Full Name
Email
Role
Status
Actions
admin
Admin
admin@cityu.local
Admin
Active
Edit
Delete
admin1
Admin2
admin1@cityu.local
Admin
Active
Edit
Delete
student1
Student One
student1@cityu.local
Student
Active
Edit
Delete
student2
Student Two
student2@cityu.local
Student
Active
Edit
Delete
student3
Student Three
student3@cityu.local
Student
Active
Edit
Delete
student4
Student Four
student4@cityu.local
Student
Active
Edit
Delete
student5
Student Five
student5@cityu.local
Student
Active
Edit
Delete
instructor1
Instructor One
instructor1@cityu.local
Instructor
Active
Edit
Delete
instructor2
Instructor Two
instructor2@cityu.local
Instructor
Active
Edit
Delete
© 2026 University Course Registration System


Home
View Master Class Schedule
View Timetable
View Student List
instructor1
Log Out
Welcome to Course Registration System
Quickly jump to the page you need.
Master Class Schedule
Browse the complete class schedule for all offerings.
Timetable
View your weekly class timetable in one place.
Student List
Check registered students and export the list.

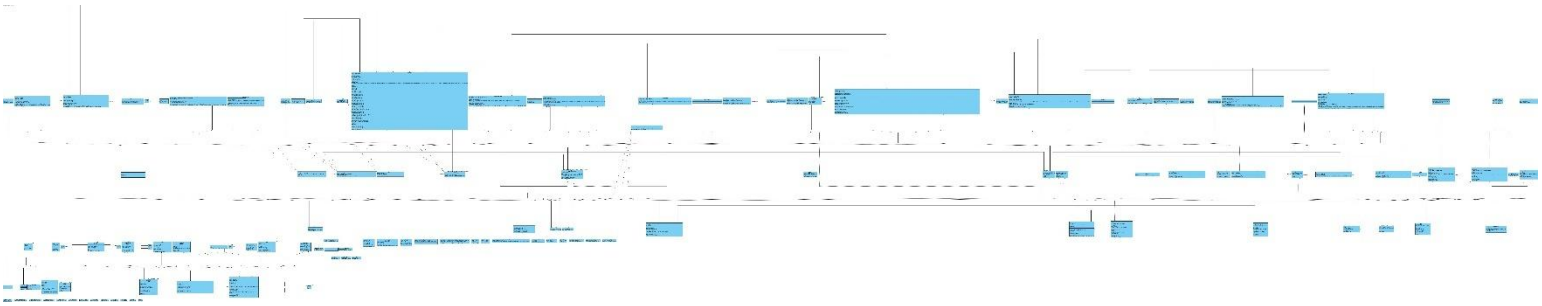

GE103 Digital Literacy Core 3 Friday 14:00 - 17:00 Online Add
GE103 Digital Literacy Core 3 Saturday 10:00 - 13:00 LT-5 Add
GE103 Digital Literacy Core 3 Wednesday 14:00 - 17:00 Online Add
GE103 Digital Literacy Core 3 Friday 14:00 - 17:00 Online Added
GE103 Digital Literacy Core 3 Saturday 10:00 - 13:00 LT-5 Added
Current Plan
localhost:8080
Selected section overlaps with GE103 (10:00-13:00). Replace overlapping entries?
Cancel
OK
Courses selected in this plan.
Code Credits Action
GE103 3 Saturday 10:00 - 13:00 Remove
GE103 3 Friday 14:00 - 17:00 Remove
Total Credits
6
Recommended: 15 to 18 credits
Below recommended range
Submit Plan
Undo All Changes
Remove Plan
Waitlist
Full courses you are waiting for.

File Persistence

Data files written by the system:

- Students.csv: persists all student fields including cohort and credit limits
- registration_records.csv: one row per enrolment with timestamp
- Sections.csv: persists all section fields
- id_sequences.csv: maintains auto-increment counters per entity

Class Diagram



Please refer to our vpp file to see the original quality diagram

The system architecture follows a layered object-oriented design consisting of controller, service, repository, and domain model classes. Controllers handle user requests, services encapsulate business logic, repositories to manage persistence, and model classes represent domain entities such as users, courses, sections, and registration records. This separation contains areas of concern into their own layers and improves maintainability, testability, and extensibility.

Software Design Principle and Patterns

To design a maintainable, extensible, and testable system, we adopted many software design principles and patterns during development. Notable patterns applied in the project include Dependency Injection, Strategy Pattern, Facade Pattern, Builder Pattern, aligning the system design with SOLID principles

Dependency Injection and Testability

Initially, a Singleton-based design was adopted for shared service and repository components such as registration service, as they provide a global access point and ensure that only one instance exists throughout the system.

However, after evaluation, we decided against using Singleton due to the following problems.

1. Tight Coupling

Singletons tightly couple us to the exact implementation of the singleton, meaning classes or modules depend on each other so strongly that changing one usually forces changes in other. This violates Single Responsibility Principle and hurts code maintainability, because it gives all dependent class more reasons to change.

2. Bad Extensibility and Testability

Singleton classes use private constructors and static access methods. This harms extensibility and testability because they cannot be extended for different implementations or unit test stubbing. This violates the Open Closed Principle as the singleton classes are not open to extension. Creating unit tests for service classes that depend on singleton repository classes is a major roadblock for improving our test coverage.

Dependency Injection

Instead of Singleton, we opted to use dependency injection, where dependents receive dependencies as arguments via constructors or methods instead of creating or getting the instance on their own. Thanks to the Spring framework, the lifetime and passing of dependencies are managed automatically, reducing human error and mental overhead.

Using dependency injection creates loosely coupled classes, so dependents do not have to know about the concrete implementation of dependencies, promoting Dependency Inversion Principle and Open Closed Principle and improves extensibility and maintainability. Test stubs become trivial to create as implementations are open to extension, improving software testability.

Strategy Pattern // Todo: update to reflect timetable code change

The Strategy Pattern was used when multiple interchangeable behaviors may exist for the same operation.

One example is timetable viewing and exporting, as there can be different desirable layout and file formats to present a timetable for different roles, such

as black and white or colored, plain text or fully rendered, compact or detailed layout, etc. Three interfaces, TimetableExporter, TimetableFormatter, and TimetableDataHandler (TimetableOwnerProvider/TimetableRoleStrategy), are created to define an interface to encapsulate the different implementations, or strategies, of exporting, formatting, and handling role-specific data for timetables.

Open Close Principle

This allows easy addition of new presentation formats for adapting to new requirements, as the system remains open for extension; while existing users remain unaffected as existing implementations are closed to modification.

Interface Segregation Principle

This design separates the concern of exporting for different file formats, formatting for different layouts, and data handling for different roles, so individual strategies remain simple and focused on one purpose, improving maintainability. This separation allows clients to ignore irrelevant implementations and only choose relevant, desired behaviors, reduce unnecessary dependencies, and improve flexibility.

Observer Pattern

Observer patterns are implemented for auto adding waitlisted students when some enrolled students drop a class.

By using an observer pattern, the system can become more maintainable and extensible. For example, if your email notification service needs to be implemented in the future, we can simply subscribe to the observer without the modification of the existing code.

This design ensures that broadcast communication from one object to other objects can be done in an easy, maintainable, and extensible way.

Adapter Pattern

The system provides different persistence mechanisms (for example CSV-based storage and JPA repositories) while keeping business logic unchanged.

To achieve this, an Adapter-style repository layer was used: service classes depend on repository port interfaces, and concrete CSV repository classes adapt low-level CSV file operations to those same interfaces.

- Interchangeable persistence implementations without changing service logic
- Decoupling between business layer and storage-specific details
    - Easier testing and future extensibility (new data source can be added by implementing the same port)

In this design, the port interface acts as the target contract, while each CSV repository acts as an adapter that translates domain-level operations (find, save, delete, query with related data) into file-based read/write behavior.

Because of this, upper layers interact with one consistent repository API and do not need to know whether data comes from a database or CSV files, adhering to Open Closed Principle and allowing different implementation of data persistence in the future.

Facade Pattern

Administrative functions often involve multiple lower-level operations such as course management, section maintenance, user updates, and data validation.

To simplify these operations, a Facade-style service component was used. For example, AdministrativeService provides a unified interface for management functions while internally coordinating repositories and supporting services.

This design reduces complexity for user interface components because they interact with one simplified service rather than many subsystems.

- Simpler interfaces for clients

- Better separation of concerns
- Reduced dependency on internal subsystem details

Builder Pattern

Some system outputs require assembling multiple pieces of data into one structured result. Examples include:

- Student timetable
- Master class schedule
- Study plan summary
- Exported timetable content
- Student enrollment list

To support clean construction of these complex outputs, the Builder Pattern was applied conceptually. A builder object can gradually assemble formatted results from course, section, and registration data.

Benefits include:

- Cleaner object construction process
- Better readability
- Separation of assembly logic from business logic
- Easier extension when output format changes

The Builder Pattern is especially useful when generated outputs contain multiple optional or repeating components.

Software Testing

- Todo
- Objective: what to test
- Design: how to test
- Execution: run tests
- Analysis: how effective is the test

To ensure the quality and accuracy of the system, we conducted continuous, automated unit testing during development, so we can programmatically verify core services correctly encapsulate business logic and requirements and gracefully handle errors and exceptions.

Our tests cases focus on registration, timetable, and administrative services and verify business logic, such as the validation logic for course section registration on existing records, course section capacity, prerequisite and exclusive courses, and time conflicts.

We use test stubs to isolate the scope of each test case, testing the behaviors independently so we can separate different areas of concern and produce consistent results that are unaffected by global state or dependencies, improving test stability and maintainability.

Mockito library for the ease of stubbing. The Mockito framework provides us with an easy way to mock dependencies' behavior. So, we can focus on testing business logic instead of spending efforts on writing hundreds of lines of manual stubbing. Take testing the addSection() logic as an example:

```
class testStub_StudentRepo implements StudentRepository {
    @Override
    public Optional<Student> findById(Integer id) {
        return Optional.of(new Student(1, "Alice", 1));
    }
    @Override public void save(Student s) {}
    @Override public void delete(Integer id) {}
    // needs to implement 10+ empty methods here
}

@Test
void addSection_Manual_Example() {
    testStub_StudentRepo fakeRepo = new testStub_StudentRepo();
    testStubSectionRepo fakeSection = testStub_SectionRepo(); // Another class to write
    RegistrationService service = new RegistrationService(fakeRepo, fakeSection, ...);
    service.addSection(1, 1, LocalDateTime.now());
}
```

Fig 3.1 Without Mockito

```
@ExtendWith(MockitoExtension.class)
public class RegistrationServiceTest {

    @Mock private StudentRepositoryPort studentRepository;
    @InjectMocks private RegistrationService registrationService;

    @Test
    void addSection_Success() {
        when(studentRepository.findById(1)).thenReturn(Optional.of(student));
        registrationService.addSection(1, 1, timestamp);
        verify(studentRepository).findById(1);
        verify(registrationRecordRepository).save(any());
    }
}
```

Fig3.2 With Mockito

Mockito also provide advantages like:

- Isolation: Replace real dependencies with mock objects.
- Readability: when(...). then(...) is human readable and intuitive compared to Manual Stubbing.
- Exception handling: Mockito API provides a convenient way to test how code throws Exceptions. Avoiding extensive amount of try {} catch {}
- Simplicity (Don't Repeat Yourself): If Mockito detects unnecessary stubbing exists in the test cases, that test cases will be flagged as failed. This forces us to remove leftover stubs and copy-pasted code from other

testcases. Keeps the testcase clean and not duplicated. (Fig3.3)

```
org.mockito.exceptions.misusing.UnnecessaryStubbingException:
Unnecessary stubbings detected.
```

Fig 3.3 Mockito throws error when there is unnecessary stubbing.

JaCoCo was used to measure test coverage and identify untested logic.

RegistrationService.addSection():

Scenario	Expected Outcome
Student not found	Exception thrown
Section not found	Exception thrown
Already enrolled	Exception thrown
Valid enrollment	Section added successfully

```
@Test
void addSectionThrowWhenSectionNotFound {
    // ...

    RuntimeException ex = assertThrows(RuntimeException.class,
        () -> service.addSection(1, 10, LocalDateTime.now()));
    assertTrue(ex.getMessage().contains("Section not found"));
}
```

Assertions were used to compare the **expected** and **actual** outcomes.

This verifies both error handling and normal execution.

TimetableService.exportTimetable():

Scenario	Expected Outcome
Exporter is null	Exception thrown
Student Record is empty	Exception thrown
Invalid student ID	Exception thrown
Valid Input	Timetable exported successfully

```
@Test
public void testExporterInvocationWithData() throws TimetableExportException {
    Path mockPath = mock(Path.class);
    when(mockExporter.export(any(TimetableData.class)))
        .thenReturn(mockPath);

    List<RegistrationRecord> records = Arrays.asList(new RegistrationRecord());
    TimetableData data = new TimetableData.Builder()
        .studentId(12345)
        .registrationRecords(records)
        .build();

    Path result = mockExporter.export(data);
    verify(mockExporter).export(data);
    assertEquals(mockPath, result);
}
```

RegistrationService.waitListSection():

```
@Test
void waitListSection_SectionNotFound() {
    Integer studentId = 1;
    Integer sectionId = 999;

    when(studentRepository.findById(studentId)).thenReturn(Optional.of(student));
    when(sectionRepository.findById(sectionId)).thenReturn(Optional.empty());

    RuntimeException exception = assertThrows(expectedType: RuntimeException.class,
            () -> registrationService.waitListSection(studentId, sectionId, timestamp));

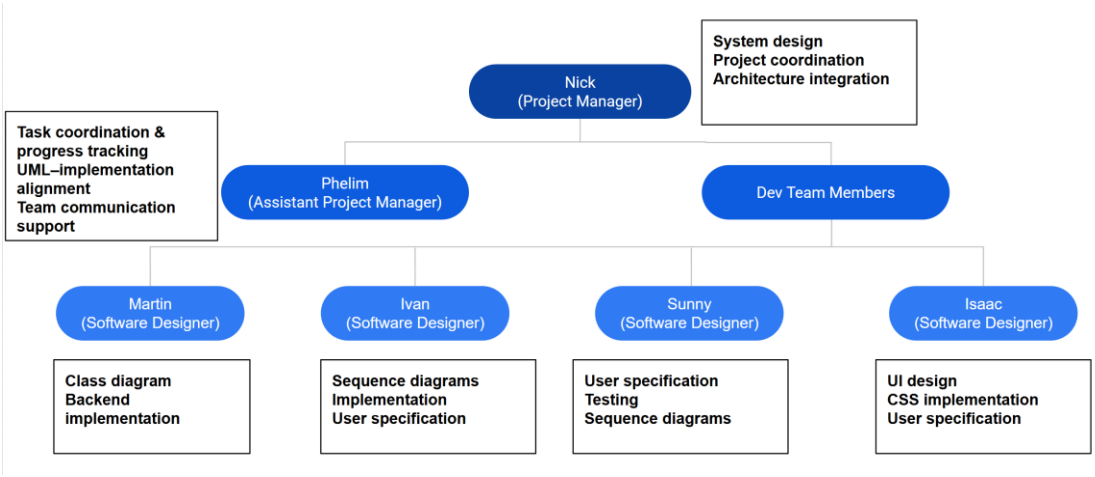
    assertEquals(expected: "Section not found", exception.getMessage());
}
```

Student does not exist	Exception thrown: **"Student not found"**
Section does not exist	Exception thrown: **"Section not found"**
Student cohort not in active	Exception thrown: **"Student not eligible to**

cohorts	**register"**
Student already enrolled in section	Exception thrown: **"Already enrolled"**
Student already waitlisted	Exception thrown: **"Already waitlisted"**
Time conflict with an enrolled section	Exception thrown: **"Time conflict with existing section"**
Time conflict with another waitlisted section	Exception thrown: **"Time conflict with waitlisted section"**
Waitlist capacity reached	Exception thrown: **"Waitlist is already full"**
Valid input and all checks passed	Waitlist record created successfully

Weekly Activity and Contribution

Work Allocation



Weekly Time Sheet and Activity

Course	CS3342 Software Design
Year	
Project No.	Group 33
Project Name/Title	Course Registration System
No. of Team Members	6
Project Manager Name	Lee Kwan Ho, Phelim

Student ID	Name (Last, First)	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Total
59284834	Lee Kwan Ho, Phelim	1	1	1	1	1	3	3	2	3	3	2	2	2	25
58527052	Yip Tung Yin, Sunny	1	1	1	1	1	3	3	2	3	3	2	2	2	25
59285554	Chen Kin Hang, Ivan	1	1	1	1	1	3	3	2	3	3	2	2	2	25
59285357	CHAN Chun Hin, Isaac	1	1	1	1	1	3	3	2	3	3	2	2	2	25
59286956	Ieong Chun Ho, Nick	1	1	1	1	1	3	3	2	3	3	2	2	2	25
59301024	Ching Chun Hei, Martin	1	1	1	1	1	3	3	2	3	3	2	2	2	25
	Total Effort	6	6	6	6	6	18	18	12	18	18	12	12	12	150

Student	Relative	Contribution	g.Group Mar	Final Adjusted
Lee Kwan Ho, Pl	1.00	100.00%	80.00	80.00
Yip Tung Yin, S	1.00	100.00%	80.00	80.00
Chen Kin Hang,	1.00	100.00%	80.00	80.00
CHAN Chun Hir	1.00	100.00%	80.00	80.00
Ieong Chun Ho,	1.00	100.00%	80.00	80.00
Ching Chun Hei,	1.00	100.00%	80.00	80.00
Max	1.00		Average	80.00
			Max	80.00
			Min	80.00

Week	Weekly Activity Log	Completed By
Week 1	Recruit Project Memebers to Form a Team.	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 2	Project topic selection, Scope definition, Task allocation	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 3	Requirement gathering Stakeholder identification	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 4	Project plan writing, Weekly activity setup, Use case drafting	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 5	Use case drafting, Class diagram drafting, Design discussion	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 6	Sequence diagram drafting, Design pattern selection, SOLID principle planning	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 7	Project structure setup, Core implementation, File handling setup	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 8	Feature implementation	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 9	Function completion, Code integration, Error handling improvement	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 10	Test case creation, Junit testing, Bug detection	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 11	Bug fixing, Test coverage improvement, Test documentation	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 12	UML finalization, Report writing	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin
Week 13	Final code review, Release packaging, Presentation preparation	Lee Kwan Ho, Phelim, Yip Tung Yin, Sunny, Chen Kin Hang, Ivan, CHAN Chun Hin, Isaac, Ieong Chun Ho, Nick ,Ching Chun Hei, Martin

Week 1

Goal: Form team, define project direction, and set up workflow.

Work completed: Confirmed Course Registration System scope, created repository, defined communication and meeting routine.

Member contributions: Nick coordinated team formation and project direction; Phelim organized task setup, communication flow, and responsibility tracking; Martin, Ivan, Sunny, and Isaac confirmed their role areas and initial ownership.

Deliverables: Team charter, initial project scope, repository initialization.

Issue and resolution: Early role overlap was identified; resolved by assigning clear module ownership and backup owners.

Week 2

Goal: Finalize project plan and task allocation.

Work completed: Broke system into modules including login/access control, registration, waitlist, timetable, admin, and testing.

Member contributions: Nick led project planning and module allocation decisions; Phelim tracked task distribution and workload balance; Martin mapped backend and class-design-related modules; Ivan and Sunny helped define use-case and interaction-related modules; Isaac identified UI-related responsibilities.

Deliverables: Milestone schedule, module-level work plan, initial timeline.

Issue and resolution: Task estimates were too optimistic; resolved by splitting large tasks into smaller weekly targets.

Week 3

Goal: Gather and structure requirements.

Work completed: Converted user needs into use cases for Student, Instructor, and Admin workflows.

Member contributions: Sunny, Ivan, and Isaac documented user requirements and role-based system needs; Nick reviewed requirement completeness against project scope; Phelim tracked requirement assignments and alignment; Martin

checked backend feasibility of the identified requirements.

Deliverables: Requirement list, use case draft set, acceptance criteria baseline.

Issue and resolution: Some requirements were ambiguous; resolved through clarification meetings and agreed definitions.

Week 4

Goal: Prepare analysis and early design artifacts.

Work completed: Drafted use case details and system interaction flow for login, add/drop, waitlist, and timetable operations.

Member contributions: Ivan and Sunny developed use case details and sequence-oriented interaction flow; Isaac refined user-facing flow from a UI perspective; Martin connected analysis outputs to backend design implications; Phelim monitored documentation alignment; Nick reviewed consistency with overall system design.

Deliverables: Updated use case documentation and flow mapping.

Issue and resolution: Edge-case behavior was unclear for failed registration attempts; resolved by documenting fallback and waitlist rules.

Week 5

Goal: Build class-level design foundation.

Work completed: Expanded domain model for Student, Instructor, Course, Section, RegistrationRecord, RegistrationPlan, and related entities.

Member contributions: Martin led class diagram and domain model refinement; Ivan and Sunny reviewed behavior flow consistency with sequence logic and user needs; Isaac checked whether the design supported interface

requirements; Phelim ensured UML artifacts remained aligned with implementation direction; Nick approved the design structure.

Deliverables: Refined class design and relationship definitions for implementation use.

Issue and resolution: Duplicate data attributes appeared across entities; resolved by normalizing shared fields and clarifying ownership.

Week 6

Goal: Finalize design strategy and architecture decisions.

Work completed: Defined layered architecture and design direction for maintainability, including separated service responsibilities and pattern usage.

Member contributions: Nick led architecture decisions and system-level design direction; Phelim coordinated design tracking and UML-to-implementation alignment; Martin contributed backend structure decisions; Ivan and Sunny supported sequence-level and interaction design planning; Isaac reviewed architectural implications for UI integration.

Deliverables: Design decision summary, implementation guidelines, architecture alignment notes.

Issue and resolution: Tight coupling risk was identified early; resolved by enforcing service and repository separation.

Week 7

Goal: Implement base project structure and persistence setup.

Work completed: Built Spring layered structure with CLI, Service, and Repository modules; established repository ports and CSV adapters.

Member contributions: Martin led backend scaffolding and repository-related implementation; Ivan supported implementation structure and interaction flow mapping; Isaac implemented initial CLI runner; Nick reviewed architectural integration across modules; Phelim tracked implementation progress and alignment.

Deliverables: Initial runnable application structure and persistence framework.

Issue and resolution: Data access flow mismatch occurred between layers; resolved by standardizing repository interfaces.

Week 8

Goal: Deliver core feature implementation.

Work completed: Implemented major functions across registration actions, plan management, admin operations, timetable flow, and CLI command support.

Member contributions: Martin and Ivan handled major implementation work for core system features; Isaac supported CLI-related feature integration; Sunny reviewed feature behavior against user requirements and prepared testing input; Nick monitored integration across modules; Phelim tracked progress and resolved coordination issues.

Deliverables: Functional feature set for major use cases and command-level operations.

Issue and resolution: Some command behaviors were missing edge cases handling; resolved by reviewing and updating business rules.

Week 9

Goal: Integrate components and improve robustness.

Work completed: Completed cross-module integration and strengthened exception handling and validation logic in registration and administrative paths.

Member contributions: Martin and Ivan worked on cross-module integration and implementation fixes; Sunny checked requirement conformance and test-oriented validation paths; Isaac reviewed UI-side consistency after integration changes; Nick supervised architecture consistency; Phelim coordinated issue tracking and follow-up actions.

Deliverables: Integrated build with improved error handling and stable workflow transitions.

Issue and resolution: Integration exposed conflicting assumptions across modules; resolved with validation rules alignment.

Week 10

Goal: Expand and execute testing strategy.

Work completed: Created and executed tests for services, repositories, and CLI session/command flows using Maven test pipeline.

Member contributions: Sunny led testing design and execution; Martin and Ivan developed and supported service, implementation, and flow-based tests; Isaac checked UI-related behavior and usability issues; Phelim coordinated bug reporting and testing progress; Nick reviewed coverage completeness and quality status.

Deliverables: Broad automated test suite and initial coverage evidence.

Issue and resolution: Some tests were flaky due to setup differences; resolved by standardizing test fixtures and data initialization.

Week 11

Goal: Bug fixing and quality refinement.

Work completed: Addressed defects found in test runs, improved edge-case handling, and increased reliability of business rule enforcement.

Member contributions: Martin and Ivan fixed implementation defects and logic issues; Sunny verified fixes through testing and regression checks; Isaac reviewed UI-related corrections; Phelim tracked bug resolution progress and communication; Nick monitored final quality refinement and stability.

Deliverables: Stabilized implementation and stronger test pass consistency.

Issue and resolution: Regression bugs appeared after fixes; resolved through targeted retesting and smaller, safer patch cycles.

Week 12

Goal: Finalize documentation and design reporting.

Work completed: Updated analysis/design documentation to match actual implemented architecture, including service refactoring and timetable export structure.

Member contributions: Nick led final documentation review for design and architecture accuracy; Phelim coordinated traceability between UML/design artifacts and implementation; Martin updated backend-related design sections; Ivan and Sunny revised sequence, specification, and testing-related documentation; Isaac updated UI-related documentation.

Deliverables: Finalized analysis and design write-up with implementation-aligned descriptions.

Issue and resolution: Documentation lagged code changes; resolved by performing section-by-section traceability checks.

Week 13

Goal: Final review, packaging, and presentation preparation.

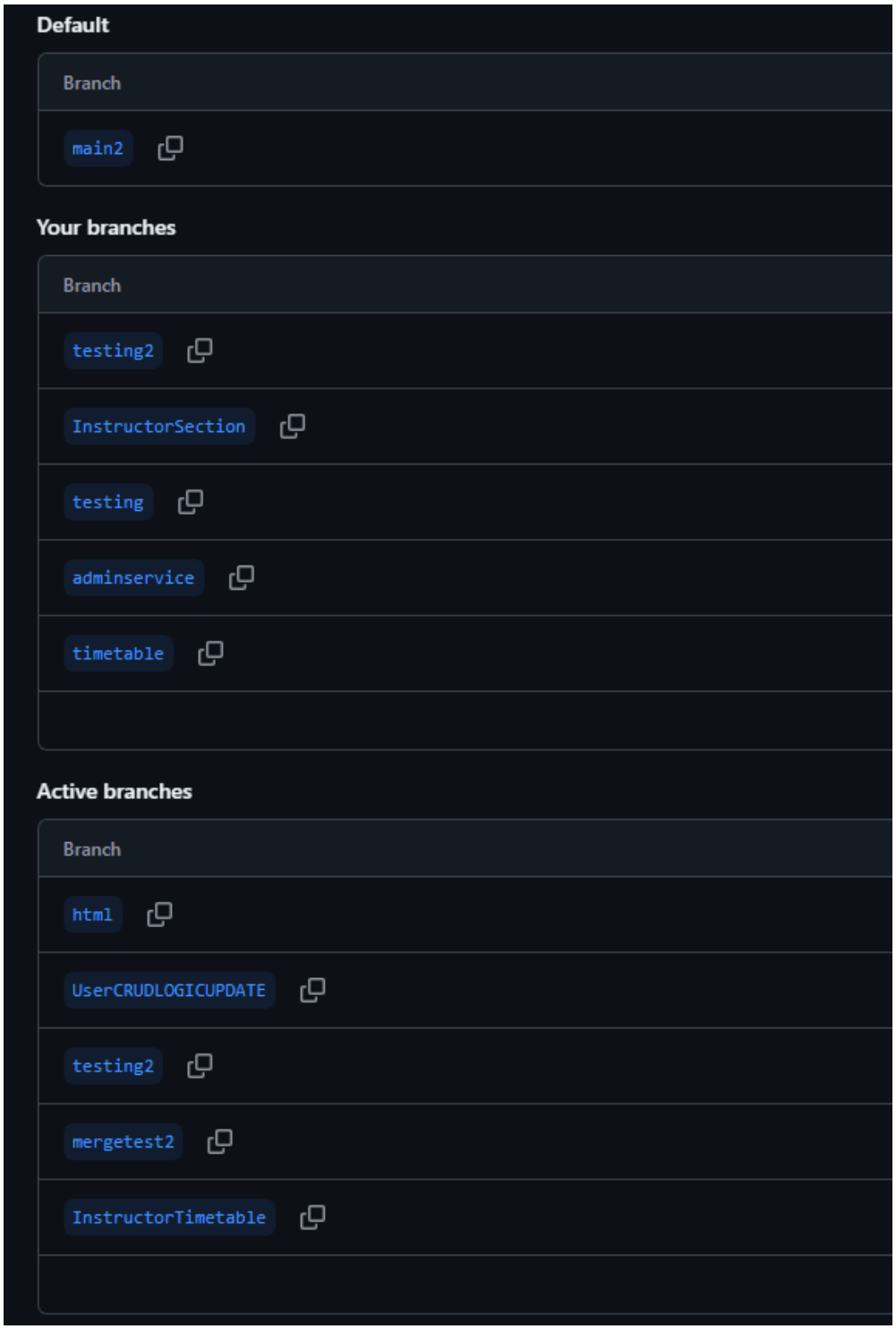
Work completed: Performed end-to-end verification, consolidated report sections, prepared demo scenarios, and finalized release package.

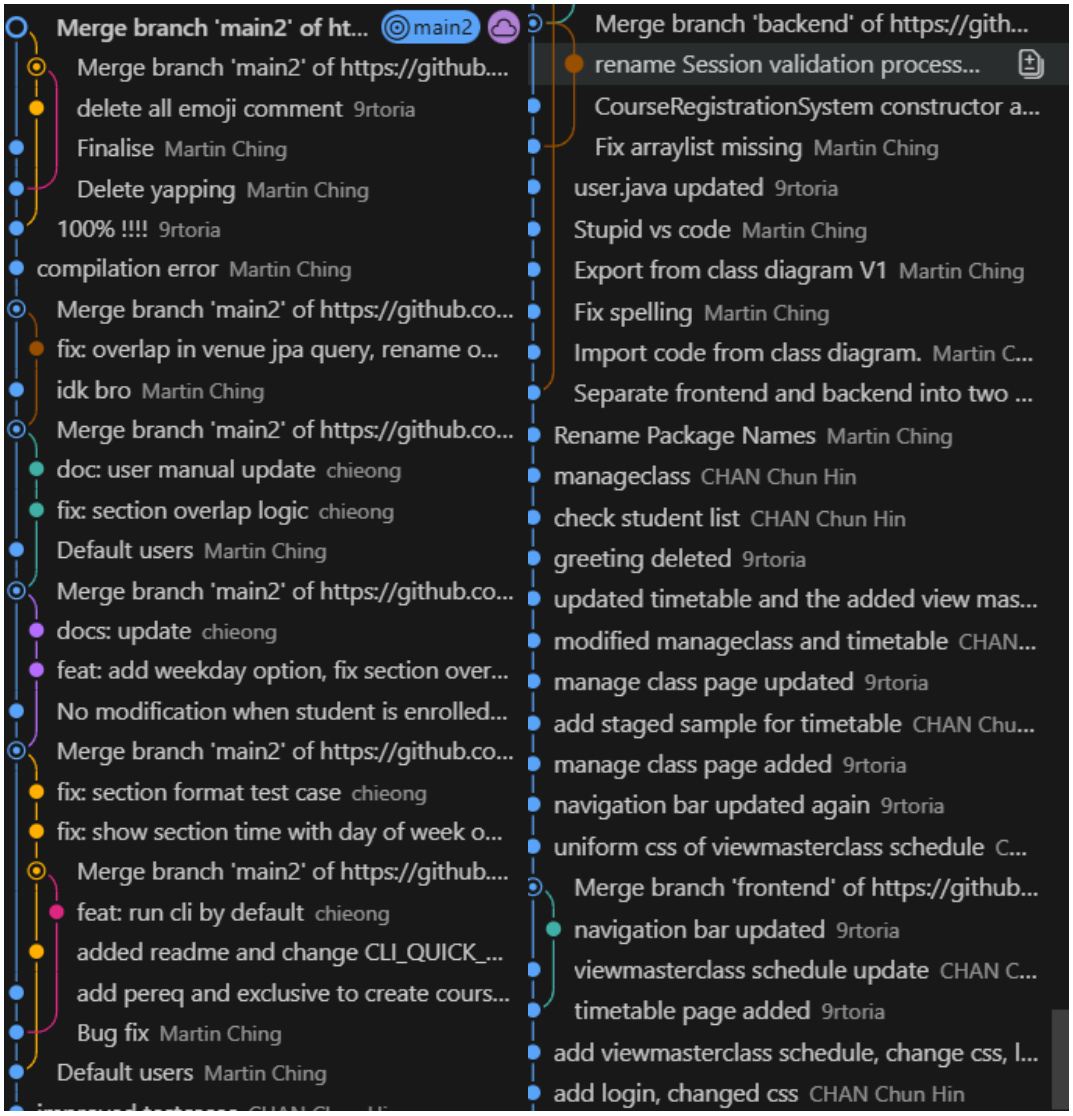
Member contributions: Nick led final documentation review for design and architecture accuracy; Phelim coordinated traceability between UML/design artifacts and implementation; Martin updated backend-related design sections; Ivan and Sunny revised sequence, specification, and testing-related documentation; Isaac updated UI-related documentation.

Deliverables: Final report package, validated demonstration flow, submission-ready project.

Issue and resolution: Final content consistency was a risk; resolved through final checklist review and cross-verification before submission.

GitHub Activity Evidence




Merge branch 'main2' of ht... main2
Merge branch 'main2' of https://github....
delete all emoji comment 9rtoria
Finalise Martin Ching
Delete yapping Martin Ching
100% !!!! 9rtoria
compilation error Martin Ching
Merge branch 'main2' of https://github.co...
fix: overlap in venue jpa query, rename o...
idk bro Martin Ching
Merge branch 'main2' of https://github.co...
doc: user manual update chieong
fix: section overlap logic chieong
Default users Martin Ching
Merge branch 'main2' of https://github.co...
docs: update chieong
feat: add weekday option, fix section over...
No modification when student is enrolled...
Merge branch 'main2' of https://github.co...
fix: section format test case chieong
fix: show section time with day of week o...
Merge branch 'main2' of https://github....
feat: run cli by default chieong
added readme and change CLI_QUICK_...
add pereq and exclusive to create cours...
Bug fix Martin Ching
Default users Martin Ching
Merge branch 'backend' of https://gith...
rename Session validation process...
CourseRegistrationSystem constructor a...
Fix arraylist missing Martin Ching
user.java updated 9rtoria
Stupid vs code Martin Ching
Export from class diagram V1 Martin Ching
Fix spelling Martin Ching
Import code from class diagram. Martin C...
Separate frontend and backend into two ...
Rename Package Names Martin Ching
manageclass CHAN Chun Hin
check student list CHAN Chun Hin
greeting deleted 9rtoria
updated timetable and the added view mas...
modified manageclass and timetable CHAN...
manage class page updated 9rtoria
add staged sample for timetable CHAN Chu...
manage class page added 9rtoria
navigation bar updated again 9rtoria
uniform css of viewmasterclass schedule C...
Merge branch 'frontend' of https://github...
navigation bar updated 9rtoria
viewmasterclass schedule update CHAN C...
timetable page added 9rtoria
add viewmasterclass schedule, change css, l...
add login, changed css CHAN Chun Hin

Conclusion

This project developed a Course Registration System that enables academic registration processes for students, instructors, and administrators.

The system implements timetable management, registration validation, section enrollment, and administrative management through a modular layered architecture. By applying appropriate software design principles and patterns such as Dependency Injection, Strategy, Facade, Builder, the system achieves better maintainability, flexibility, and testability.

Continuous testing and collaborative development practices further improved system quality. The final prototype demonstrates that the proposed solution is practical, extensible, and suitable as a modern academic registration platform.